Лабораторна робота № 5

Географічні дані OpenStreetMap: від тегів до маршруту

Інтелектуальний аналіз та візуалізація даних · 4 години · 4 бали

Постановка

Ви — аналітик у туристичному застосунку. Завдання: для заданого міста **відібрати десять найвартісніших історико-культурних об'єктів** і **спланувати піший маршрут**, який починається на залізничному вокзалі, обходить усі десять і повертається на вокзал.

Слово «найвартісніші» в даних не записане. Критерій ви побудуєте самі — і захистите його.

Потім те саме для **тридцяти** об'єктів. Результат цієї, більшої задачі стане базою порівняння в **лабораторній роботі № 8**, де ту саму задачу розв'язуватимуть генетичним та мурашиним алгоритмами.

Що здаєте

1. Цей ноутбук із виконаними комірками (усі виводи збережені).
2. Файл `lr5_результати.json` — результати для ЛР8.
3. Карту `lr5_маршрут.html`.
4. Звіт у кінці ноутбука: відповіді на сім питань.

Що оцінюється

критерій	0	2	3	4
Відбір об'єктів	теги взято навмання	фільтр працює	фільтр обґрунтовано	показано, що відкинуто і чому
Критерій рейтингу	немає	є формула	ваги пояснено	порівняно з альтернативним критерієм
Маршрут	не будується	будується	перевірено коректність	пояснено різницю методів числом
Карта	немає	точки на карті	маршрут по ребрах графа	видно різницю з прямими відрізками

критерій	0	2	3	4
Звіт	немає	формально	з числами	з висновком, якого не було в завданні

---

0. Ваш варіант

Впишіть прізвище та номер у списку групи. Місто підтягнеться само.

Хід виконання. Вписуються прізвище і номер у списку групи. За номером зі списку з 30 міст автоматично визначається моє місто: номер 3 — Харків. Перевірка не дає запустити роботу з незаповненими полями.

In [1]:
```
STUDENT_NAME = 'Бобринок Ангеліна'     # група ШІ-32
LIST_NUMBER  = 3                       # номер у списку групи

CITIES = [
    'Львів', 'Київ', 'Харків', 'Одеса', 'Дніпро',
    'Чернівці', 'Івано-Франківськ', 'Тернопіль', 'Ужгород', 'Луцьк',
    'Рівне', 'Хмельницький', 'Вінниця', 'Житомир', 'Чернігів',
    'Черкаси', 'Полтава', 'Суми', 'Кропивницький', 'Запоріжжя',
    'Миколаїв', 'Кривий Ріг', 'Кам’янець-Подільський', 'Мукачево',
    'Коломия', 'Дрогобич', 'Біла Церква', 'Ніжин', 'Острiг', 'Умань',
]

assert STUDENT_NAME and LIST_NUMBER > 0, 'впишіть прізвище та номер у списку
CITY = CITIES[(LIST_NUMBER - 1) % len(CITIES)] + ', Україна'
print('варіант %d: %s' % (LIST_NUMBER, CITY))
```

```
варіант 3: Харків, Україна
```

> Якщо викладач призначив вам інше місто — просто перепишіть `CITY` вручну і зазначте це у звіті.

Хід виконання. Перевіряється, чи встановлено всі потрібні бібліотеки: для завантаження даних OpenStreetMap, роботи з геоданими, інтерактивних карт, шестикутної сітки H3 і графів. Якщо чогось бракує, воно встановлюється автоматично. У мене все вже було встановлено.

In [2]:
```
import importlib.util, subprocess, sys
REQUIRED = ['osmnx', 'geopandas', 'shapely', 'pyproj', 'pyarrow',
            'folium', 'mapclassify', 'h3', 'networkx']
missing = [p for p in REQUIRED if importlib.util.find_spec(p) is None]
if missing:
    print('встановлюю:', ', '.join(missing), '...')
    subprocess.check_call([sys.executable, '-m', 'pip', 'install', '-q', *mi
print('усі пакети на місці')
```

```
усі пакети на місці
```

Хід виконання. Підключаються бібліотеки, з якими працює вся робота: таблиці, геодані, графи, графіки та інтерактивні карти. Обирається підкладка для карти — сіра карта Esri, бо CARTO тепер вимагає ключ доступу.

In [3]:
```
import warnings; warnings.filterwarnings('ignore')
import json, time, os, collections
import numpy as np
import pandas as pd
import geopandas as gpd
import networkx as nx
import matplotlib.pyplot as plt
import folium
from xyzservices import providers as xyz

BASEMAP = xyz.Esri.WorldGrayCanvas   # CARTO з 2026 р. вимагає ключ API
pd.set_option('display.width', 200)
print('geopandas', gpd.__version__)
```

```
geopandas 1.2.0
```

---

1. Два способи дістати дані

Робоча програма вимагає порівняти два способи доступу **за обсягом переданих даних і за часом**:

- **живий запит** — `osmnx` питає Overpass API просто зараз;
- **знімок** — ті самі дані, збережені у файли GeoParquet, читаються з диска.

Нижче — живий запит. Запустіть його **один раз**: він збереже теку зі знімком, і всі наступні запуски читатимуть з неї.

> Overpass — безкоштовний сервіс і обмежує частоту. Якщо запит не пройшов — зачекайте хвилину й повторіть. Якщо не проходить взагалі, візьміть резервний знімок у викладача і вкажіть це у звіті.

Хід виконання. Налаштовується osmnx — інструмент, що завантажує дані з OpenStreetMap. Створюється тека `osm_харків`, у яку зберігаються всі шари (це і є знімок). Задається довгий час очікування відповіді, бо сервер повільний. Також визначається, які туристичні позначки вважаються пам'ятками: музей, атракція, галерея, арт-об'єкт, оглядова точка.

In [4]:
```
import osmnx as ox

OUT = 'osm_' + CITY.split(',')[0].strip().lower().replace(' ', '_')
os.makedirs(OUT, exist_ok=True)
ox.settings.use_cache = True
```

```
ox.settings.cache_folder = os.path.join(OUT, 'cache')
ox.settings.requests_timeout = 300
ox.settings.overpass_rate_limit = True

TOURISM_OK = ['museum', 'attraction', 'gallery', 'artwork', 'viewpoint']

def cache_bytes():
    import pathlib
    p = pathlib.Path(ox.settings.cache_folder)
    return sum(f.stat().st_size for f in p.glob('*.json')) if p.exists() els

print('тека знімка:', OUT)
```

```
тека знімка: osm_харків
```

Дві налаштовані дрібниці, без яких запит не пройде

Це не формальності — кожна з них ламає збір по-своєму, і жодна не схожа на свою справжню причину.

Представитись. Overpass відповідає **HTTP 406** клієнтові, який прийшов із типовим заголовком `python-requests`. `osmnx` надсилає свій `User-Agent` сам, але якщо ви звертаєтесь до сервісу напряму — надсилайте теж. Рядок має бути **лише латиницею**: HTTP-заголовки передаються в latin-1, і кирилиця кладе запит помилкою `UnicodeEncodeError` ще до відправлення.

Обрати машину. За іменем `overpass-api.de` стоїть кілька серверів, і частина з них відмовляє у з'єднанні. `osmnx` закріплює одну адресу на всю сесію — навмисно, щоб не порушувати облік слотів. Якщо закріпиться та, що відмовляє, впаде весь збір. Тому адресу обираємо самі, вже перевірену.

Хід виконання. Сервер Overpass відхиляє запити без підпису програми, тому задається підпис. Він написаний латиницею, бо кирилиця в заголовках запиту не працює. Крім того, з кількох серверів за однією адресою вибирається той, що справді відповідає. Знімок у мене вже збережено, тому мережа не знадобилась і перевірка серверів не запускалась.

In [5]:
```
import socket

USER_AGENT = ('LvivPolytechnic-SK26-coursework/1.0 '
              '(https://lpnu.ua; student lab assignment)')
assert USER_AGENT.isascii(), 'HTTP-заголовки — latin-1, кирилиця тут не прац
ox.settings.http_user_agent = USER_AGENT


def pin_working_ip(hostname, port=443, timeout=8):
    """Закріпити для цього імені адресу, яка справді відповідає."""
    try:
        infos = socket.getaddrinfo(hostname, port, socket.AF_INET, socket.SO
    except Exception as e:
        print('не вдалось резолвити %s: %s' % (hostname, type(e).__name__))
```

```
        return None
    ips = sorted({i[4][0] for i in infos})
    good = []
    for ip in ips:
        s = socket.socket(); s.settimeout(timeout)
        try:
            s.connect((ip, port)); good.append(ip)
        except Exception:
            pass
        finally:
            s.close()
    print('%s: машин %d, відповідають %d %s' % (hostname, len(ips), len(gooc
    if not good:
        return None
    original = socket.gethostbyname
    socket.gethostbyname = lambda h: good[0] if h == hostname else original(
    print('закріплено', good[0])
    return good[0]


# Знімок уже є — мережа не потрібна взагалі, нічого не перевіряємо.
if os.path.exists(os.path.join(OUT, 'road_edges.parquet')):
    print('знімок у теці %s уже є — Overpass не потрібен' % OUT)
elif os.environ.get('OVERPASS_URL'):
    # дзеркало, яке виявилось найшвидшим у probe_mirrors.py
    ox.settings.overpass_url = os.environ['OVERPASS_URL'].rstrip('/')
    pin_working_ip(ox.settings.overpass_url.split('/')[2])
    print('беру дзеркало', ox.settings.overpass_url)
elif pin_working_ip('overpass-api.de') is None:
    # запасні дзеркала зі сторінки Overpass API у вікі OSM (без російських)
    for alt in ['https://overpass.private.coffee/api', 'https://overpass.kum
        if pin_working_ip(alt.split('/')[2]):
            ox.settings.overpass_url = alt
            print('беру дзеркало', alt)
            break
```

```
знімок у теці osm_харків уже є — Overpass не потрібен
```

> Якщо жодна машина не відповіла — виконайте комірку ще раз за хвилину, або візьміть у викладача резервний знімок і зазначте це у звіті. Інші дзеркала перелічені на сторінці Overpass API; адресу можна підставити у `ox.settings.overpass_url`.

Завдання 1.1

Допишіть функцію `fetch`: вона має зберегти шар у `.parquet`, а якщо файл уже є — прочитати його, **не звертаючись до мережі**. Поверніть також, скільки МБ пішло мережею і скільки секунд це зайняло.

Підказка щодо `clean`: у OSM одна колонка може містити і число, і список (ребро, склеєне з кількох шляхів). Parquet вимагає один тип на колонку, тому неоднорідну колонку треба звести до тексту **цілком**.

Хід виконання. Описується функція, яка отримує один шар даних. Якщо шар уже збережено на диску, він читається звідти без інтернету. Якщо ні, дані завантажуються, зберігаються у файл і записується, скільки мегабайтів передано і скільки секунд це зайняло. Ці цифри потрібні, щоб порівняти два способи отримання даних. Перед збереженням колонки, де в одному рядку число, а в іншому список, перетворюються на текст, бо формат parquet вимагає одного типу в колонці. Невдалі запити повторюються кілька разів із паузою.

In [6]:
```
def clean(gdf):
    g = gdf.reset_index()
    for c in g.columns:
        if c == 'geometry':
            continue
        boxed = g[c].map(lambda v: isinstance(v, (list, dict, set, tuple))).
        if boxed or g[c].dtype == object:
            g[c] = g[c].map(
                lambda v: None if v is None or (isinstance(v, float) and pd.
                else (json.dumps(v, ensure_ascii=False)
                      if isinstance(v, (list, dict, set, tuple)) else str(v)
            ).astype('string')
    return g


COST = {}
LIVE_P = os.path.join(OUT, 'cost_live.json')   # живі цифри зберігаються одр


def save_live(name, row):
    live = json.load(open(LIVE_P, encoding='utf-8')) if os.path.exists(LIVE_
    live[name] = row
    json.dump(live, open(LIVE_P, 'w', encoding='utf-8'), ensure_ascii=False,


def with_retry(fn, attempts=4, wait=45):
    """Дзеркала Overpass обривають з'єднання або віддають 429/504 — чекаємо
    for k in range(1, attempts + 1):
        try:
            return fn()
        except Exception as e:
            if k == attempts:
                raise
            print('   спроба %d: %s — чекаю %d с' % (k, str(e)[:90], wait))
            time.sleep(wait)


USE_PBF = False                 # перемикається в комірці «резервне джерело»
SOURCE = 'живий запит'


def fetch(name, getter):
    """Повертає шар. Якщо файл уже збережено — читає з диска."""
    path = os.path.join(OUT, name + '.parquet')
```

```
    if os.path.exists(path) and os.path.getsize(path) > 0:
        t0 = time.time()
        g = gpd.read_parquet(path)
        COST[name] = {'спосіб': 'знімок', 'записів': len(g),
                      'передано_МБ': 0.0,
                      'файл_МБ': round(os.path.getsize(path) / 1e6, 3),
                      'секунд': round(time.time() - t0, 3)}
        return g

    # ----- живий запит -----
    b0, t0 = cache_bytes(), time.time()
    raw = with_retry(getter)              # тут іде запит до Nominatim / Over
    clean(raw).to_parquet(path, index=False)
    g = gpd.read_parquet(path)            # повертаємо те саме, що прочитає з
    COST[name] = {'спосіб': SOURCE, 'записів': len(g),
                  'передано_МБ': round((cache_bytes() - b0) / 1e6, 3),
                  'файл_МБ': round(os.path.getsize(path) / 1e6, 3),
                  'секунд': round(time.time() - t0, 3)}
    save_live(name, COST[name])
    return g
```

Хід виконання. Першою отримується межа міста Харків — живим запитом до сервісу Nominatim. Від неї залежать усі інші шари, бо дані беруться лише всередині цієї межі. Площа міста — 353,5 км².

In [7]:
```
boundary = fetch('boundary', lambda: ox.geocode_to_gdf(CITY))
poly = boundary.geometry.iloc[0]
print('межа міста отримана, площа %.1f км²'
      % (boundary.to_crs(32635).area.iloc[0] / 1e6))
```

```
межа міста отримана, площа 353.5 км²
```

Резервне джерело: витяг OSM у форматі `.osm.pbf`

Що сталося 01.11.2026. Живий запит до Overpass не вдався з жодного дзеркала (перевірка — `probe_mirrors.py`, із VPN і без нього):

дзеркало	відповідь
overpass-api.de, z.overpass-api.de	HTTP 502 (сервер лежить)
lz4.overpass-api.de	таймаут 30 с
overpass.private.coffee	таймаут; у першій спробі віддав *лише* пам'ятки — за ~29 хв, потім обірвав з'єднання
overpass.kumi.systems	таймаут 30 с
overpass.osm.jp	помилка SSL

Межа міста прийшла живим запитом (Nominatim працює), а решту шарів беру з **регіонального витягу OSM** — `kharkiv_oblast-latest.osm.pbf` з download.openstreetmap.fr, станом на 01.11.2026 01:22 UTC. Модуль

`pbf_snapshot.py` вирізає з нього прямокутник міста у два файли OSM XML, а далі працюють **ті самі** функції osmnx (`features_from_xml`, `graph_from_xml`) з **тими самими** тегами і тим самим фільтром пішохідної мережі, що й у `graph_from_polygon(network_type='walk')`. Коректність перевірено нижче: шар пам'яток із pbf порівнюється з тим, що встиг віддати Overpass.

Хід виконання. Сервер Overpass 1 листопада не працював на жодному дзеркалі, тому підключено резервне джерело. Завантажується готовий файл з усіма даними OSM Харківської області (54 МБ), і з нього вирізається прямокутник міста. Час завантаження й обробки теж записується. Під час повторного запуску всі шари вже лежать на диску, тому ця клітинка нічого не робить.

In [8]:
```
import pbf_snapshot, requests

PBF_URL  = 'https://download.openstreetmap.fr/extracts/europe/ukraine/kharki
PBF_PATH = os.path.join('osm_src', 'kharkiv_oblast.osm.pbf')
XML = {k: os.path.join(OUT, 'xml', k + '.osm') for k in ('features', 'walk')
LAYERS = ['heritage', 'stations', 'amenities', 'road_nodes', 'road_edges']
need_layers = not all(os.path.exists(os.path.join(OUT, f + '.parquet')) for

USE_PBF = True                      # Overpass недоступний — див. таблицю вищ
SOURCE = 'резервний pbf'

if USE_PBF and need_layers:
    os.makedirs('osm_src', exist_ok=True); os.makedirs(os.path.join(OUT, 'xm
    if not os.path.exists(PBF_PATH):
        t0 = time.time()
        with requests.get(PBF_URL, headers={'User-Agent': USER_AGENT}, strea
            r.raise_for_status()
            with open(PBF_PATH, 'wb') as f:
                for chunk in r.iter_content(1 << 20):
                    f.write(chunk)
        COST['pbf області'] = {'спосіб': 'завантаження pbf', 'записів': None
                               'передано_МБ': round(os.path.getsize(PBF_PATH
                               'файл_МБ': round(os.path.getsize(PBF_PATH) /
                               'секунд': round(time.time() - t0, 1)}
        save_live('pbf області', COST['pbf області'])
    t0 = time.time()
    XML, pbf_stats = pbf_snapshot.build(PBF_PATH, poly.bounds, os.path.join(
    COST['вирізання з pbf'] = {'спосіб': 'обробка pbf', 'записів': None, 'пе
                               'файл_МБ': round(pbf_stats['features_МБ'] + p
                               'секунд': round(time.time() - t0, 1)}
    save_live('вирізання з pbf', COST['вирізання з pbf'])
    print('вирізано з pbf:', pbf_stats)
else:
    print('усі шари вже збережено у %s — pbf не потрібен' % OUT)
```

```
усі шари вже збережено у osm_харків — pbf не потрібен
```

Хід виконання. Завантажуються чотири шари: пам'ятки (історичні й туристичні об'єкти), вокзали і станції, заклади (кафе, ресторани, питна вода, туалети) і пішохідна мережа міста. Використовуються ті самі позначки й той самий фільтр

пішохідних доріг, що й під час живого запиту. Мережа має 65 588 вузлів (перехресть) і 189 104 ребра (відрізки доріг).

In [9]:
```
# ті самі теги; джерело — Overpass або локальний XML, вирізаний з pbf
def features(tags):
    if USE_PBF:
        return lambda: ox.features_from_xml(XML['features'], polygon=poly, t
    return lambda: ox.features_from_polygon(poly, tags)

heritage  = fetch('heritage',  features({'historic': True, 'tourism': TOURIS
stations  = fetch('stations',  features({'railway': ['station', 'halt'],
                                         'building': ['train_station']}))
amenities = fetch('amenities', features({'amenity': ['cafe', 'restaurant',
                                                     'drinking_water', 'toil

# Мережа приходить двома таблицями одразу, тому їй окрема гілка.
NODES_P = os.path.join(OUT, 'road_nodes.parquet')
EDGES_P = os.path.join(OUT, 'road_edges.parquet')

if os.path.exists(NODES_P) and os.path.exists(EDGES_P):
    t0 = time.time()
    nodes, edges = gpd.read_parquet(NODES_P), gpd.read_parquet(EDGES_P)
    COST['мережа walk'] = {'спосіб': 'знімок', 'записів': len(nodes) + len(e
                           'передано_МБ': 0.0,
                           'файл_МБ': round((os.path.getsize(NODES_P)
                                             + os.path.getsize(EDGES_P)) / 1
                           'секунд': round(time.time() - t0, 3)}
else:
    t0, b0 = time.time(), cache_bytes()
    if USE_PBF:
        G_raw = pbf_snapshot.walk_graph(XML['walk'], poly)   # той самий філ
    else:
        G_raw = with_retry(lambda: ox.graph_from_polygon(poly, network_type=
    nodes, edges = ox.convert.graph_to_gdfs(G_raw)
    clean(nodes).to_parquet(NODES_P, index=False)
    clean(edges).to_parquet(EDGES_P, index=False)
    nodes, edges = gpd.read_parquet(NODES_P), gpd.read_parquet(EDGES_P)
    COST['мережа walk'] = {'спосіб': SOURCE, 'записів': len(nodes) + len(edg
                           'передано_МБ': round((cache_bytes() - b0) / 1e6,
                           'файл_МБ': round((os.path.getsize(NODES_P)
                                             + os.path.getsize(EDGES_P)) / 1
                           'секунд': round(time.time() - t0, 1)}
    save_live('мережа walk', COST['мережа walk'])

print('вузлів %d | ребер %d' % (len(nodes), len(edges)))
```

```
вузлів 65588 | ребер 189104
```

Завдання 1.2

Зведіть `COST` у таблицю і дайте відповідь у звіті: **у скільки разів читання знімка швидше за живий запит** і **скільки мегабайтів ви не передали мережею** при повторному запуску.

Контрольна ознака правильності: при **другому** запуску ноутбука всі рядки мають мати спосіб «знімок», а `передано_МБ` — нуль.

Хід виконання. Записані час і обсяги зводяться в таблицю. Порівнюється перший запуск, коли дані завантажувались, із повторним, коли вони читаються з диска. Повторний запуск у сотні разів швидший, і мережею не треба передавати 54,6 МБ. Контрольна ознака виконується: при повторному запуску всі шари мають спосіб «знімок» і нуль переданих мегабайтів.

In [10]:
```
cost = pd.DataFrame(COST).T
display(cost)

# Живі цифри кожен шар записав у cost_live.json у момент завантаження,
# тож порівняння видно й при повторному запуску, коли всі рядки — «знімок».
if os.path.exists(LIVE_P) and (cost['спосіб'] == 'знімок').all():
    live = pd.DataFrame(json.load(open(LIVE_P, encoding='utf-8'))).T
    cost = cost.reindex(live.index)
    cmp_ = pd.DataFrame({
        'перший запуск, с': live['секунд'].astype(float),
        'знімок, с': cost['секунд'].astype(float),
        'передано першим, МБ': live['передано_МБ'].astype(float),
        'передано знімком, МБ': cost['передано_МБ'].astype(float),
    })
    cmp_.insert(0, 'перший запуск: спосіб', live['спосіб'])
    cmp_['швидше, разів'] = (cmp_['перший запуск, с'] / cmp_['знімок, с'].cl
    display(cmp_)
    tot_live, tot_snap = cmp_['перший запуск, с'].sum(), cmp_['знімок, с'].f
    print('усього: перший запуск %.1f с, знімок %.2f с -> швидше в %.0f разі
          % (tot_live, tot_snap, tot_live / tot_snap))
    print('не передано мережею при повторному запуску: %.2f МБ'
          % cmp_['передано першим, МБ'].sum())
else:
    print('у цьому запуску є живі запити; порівняння з’явиться при повторном
```

	спосіб	записів	передано_МБ	файл_МБ	секунд
boundary	знімок	1	0.0	0.046	0.137
heritage	знімок	807	0.0	0.197	0.017
stations	знімок	88	0.0	0.07	0.013
amenities	знімок	1383	0.0	0.175	0.016
мережа walk	знімок	254692	0.0	12.554	0.078

	перший запуск: спосіб	перший запуск, с	знімок, с	передано першим, МБ	передано знімком, МБ	швидше, разів
boundary	живий запит	1.353	0.137	0.114	0.0	10.0
pbf області	завантаження pbf	42.200	NaN	54.464	NaN	NaN
вирізання з pbf	обробка pbf	67.900	NaN	0.000	NaN	NaN
heritage	резервний pbf	0.173	0.017	0.000	0.0	10.0
stations	резервний pbf	0.143	0.013	0.000	0.0	11.0
amenities	резервний pbf	0.152	0.016	0.000	0.0	10.0
мережа walk	резервний pbf	14.400	0.078	0.000	0.0	185.0

```
усього: перший запуск 126.3 с, знімок 0.26 с -> швидше в 484 разів
не передано мережею при повторному запуску: 54.58 МБ
```

Хід виконання. Перевіряється, чи можна довіряти резервному джерелу. Пам'ятки з файлу області порівнюються з тими, що встиг віддати сервер Overpass під час першої спроби. Спільних об'єктів 791 приблизно з 800. Розбіжність невелика і пояснюється тим, що дані взято в різний час.

In [11]:
```
# Перевірка резервного джерела: пам'ятки з pbf проти тих, що встиг віддати O
OLD = 'osm_харків_спроба1/heritage.parquet'
if os.path.exists(OLD):
    ov = gpd.read_parquet(OLD)
    key = lambda g: set(zip(g['element'], g['id'].astype('int64')))
    a, b = key(ov), key(heritage)
    print('Overpass (01.11, 20:10) : %d об’єктів' % len(ov))
    print('pbf      (01.11, 01:22) : %d об’єктів' % len(heritage))
    print('спільних за (тип, id)   : %d | лише в Overpass: %d | лише в pbf:
          % (len(a & b), len(a - b), len(b - a)))
    extra = heritage[[k in (b - a) for k in key(heritage)]]
    print('лише в pbf (приклади)   :', list(extra['name'].dropna().head(6)))
```

```
Overpass (01.11, 20:10) : 796 об’єктів
pbf      (01.11, 01:22) : 807 об’єктів
спільних за (тип, id)   : 791 | лише в Overpass: 5 | лише в pbf: 16
лише в pbf (приклади)   : ['Алчевському Олексію Кириловичу', 'Стенд Героям У
країни', "Пам'ятник на честь 200-річчя Харківської єпархії", "Пам'яник лікві
даторам Чорнобильської аварії", 'Лопанська стрілка', 'Ніса (Фургон, експона
т)']
```

Хід виконання. Показується збережена таблиця з цифрами першого запуску. Так порівняння видно навіть тоді, коли зараз усе читається з диска.

In [12]:
```
display(pd.read_json(os.path.join(OUT, 'cost_live.json'), orient='index'))
```

	спосіб	записів	передано_МБ	файл_МБ	секунд
boundary	живий запит	1.0	0.114	0.046	1.353
pbf області	завантаження pbf	NaN	54.464	54.464	42.200
вирізання з pbf	обробка pbf	NaN	0.000	60.790	67.900
heritage	резервний pbf	807.0	0.000	0.197	0.173
stations	резервний pbf	88.0	0.000	0.070	0.143
amenities	резервний pbf	1383.0	0.000	0.175	0.152
мережа walk	резервний pbf	254692.0	0.000	12.554	14.400

> **Чому пішохідна мережа, а не автомобільна.** У старому місті музеї стоять за сотню метрів один від одного, а автомобільних вузлів там одиниці. Кілька пам'яток сядуть на **один** вузол графа, і матриця віддалей скаже, що між ними нуль метрів. Задача розв'яжеться, відповідь буде неправильною, і повідомлення про помилку не з'явиться. У кроці 4 ви це перевірите явно.

---

2. Хто претендує на звання пам'ятки

У OpenStreetMap немає тега «визначна пам'ятка». Є два сімейства:

- `historic=*` — `castle`, `church`, `monument`, `memorial`, `ruins`, `tower`, `yes` …
- `tourism=*` — `museum`, `gallery`, `attraction`, `artwork`, `viewpoint`

Перетин неповний: музей у сучасній будівлі має лише `tourism`, руїни фортеці — лише `historic`. Тому беремо **об'єднання**.

📖 https://wiki.openstreetmap.org/wiki/Map_Features

Завдання 2.1

Відберіть кандидатів і **назвіть у звіті, скільки об'єктів ви відкинули й чому**.

Хід виконання. Відбираються кандидати в пам'ятки: об'єкти з історичною або туристичною позначкою. Відкидаються об'єкти без назви (95), бо в маршруті не може бути «якась оглядова точка». Відкидаються також меморіальні дошки, межові камені й позначки (110), бо це не місця, куди йдуть пішки. Для кожної причини виводиться, скільки об'єктів відкинуто, і приклади. Лишається 602 кандидати.

In [13]:
```
has_hist = heritage['historic'].notna()
has_tour = heritage['tourism'].isin(TOURISM_OK)
has_name = heritage['name'].notna() & (heritage['name'].str.strip() != '')
mask = has_hist | has_tour

# Дрібні об'єкти, які мають тег historic, але не є місцем, куди йдуть пішки:
# меморіальні дошки на стінах, межові та кілометрові камені, позначки.
memorial_kind = heritage['memorial'] if 'memorial' in heritage else pd.Serie
NOT_A_STOP = {'boundary_stone', 'milestone', 'marker', 'district', 'road', '
minor = memorial_kind.isin(['plaque', 'blue_plaque', 'stolperstein']) | \
        heritage['historic'].isin(NOT_A_STOP)

cand = heritage[mask & has_name & ~minor].reset_index(drop=True)

print('усього у шарі heritage :', len(heritage))
print('підходять за тегами    :', int(mask.sum()))
print('з них мають name       :', int((mask & has_name).sum()))
print('з них не дошки/камені  :', len(cand))

rejected = pd.DataFrame({
    'причина': ['не historic і не tourism з переліку', 'немає name',
                'меморіальна дошка / межовий камінь / позначка'],
    'відкинуто': [int((~mask).sum()), int((mask & ~has_name).sum()),
                  int((mask & has_name & minor).sum())],
})
display(rejected)

print('що саме було серед безіменних (historic / tourism):')
noname = heritage[mask & ~has_name]
print(noname['historic'].fillna('—').value_counts().head(8).to_string())
print(noname['tourism'].fillna('—').value_counts().head(5).to_string())
print()
print('серед «дрібних»:')
print(heritage[mask & has_name & minor]['historic'].value_counts().head(6).t
print('приклади:', list(heritage[mask & has_name & minor]['name'].head(5)))
```

```
усього у шарі heritage : 807
підходять за тегами    : 807
з них мають name       : 712
з них не дошки/камені  : 602
```

	причина	відкинуто
0	не historic і не tourism з переліку	0
1	немає name	95
2	меморіальна дошка / межовий камінь / позначка	110

```
що саме було серед безіменних (historic / tourism):
historic
—           61
memorial    17
yes          4
ruins        4
monument     2
vehicle      2
tomb         2
building     2
tourism
—           34
viewpoint   33
artwork     26
museum       1
gallery      1

серед «дрібних»:
historic
memorial    110
приклади: ['Отакару Ярошу', 'Дерев’янка вулиця', 'Алея трудової слави', "Па
м'ятна дошка В. Короленко", "Пам'ятна дошка Івашко Володимиру Антоновичу"]
```

Хід виконання. Перевіряється, з чого складається список кандидатів: які типи об'єктів у ньому є і скільки серед них точок і полігонів. Мають бути і ті, й інші: пам'ятники в OSM позначені точками, а музеї — будівлями. Є 516 точок і 86 полігонів, тож фільтр нічого зайвого не викинув.

In [14]:
```
assert 'cand' in dir(), 'спершу виконайте завдання 2.1'
print('=== historic ===');  print(cand['historic'].value_counts().head(10).t
print()
print('=== tourism ===');   print(cand['tourism'].value_counts().head(10).to
print()
print('типи геометрії:', cand.geometry.geom_type.value_counts().to_dict())
```

```
=== historic ===
historic
memorial       157
aircraft        33
monument        24
locomotive      15
yes             10
railway_car     10
tank             9
cannon           8
vehicle          5
building         4

=== tourism ===
tourism
artwork       186
attraction     92
museum         36
gallery        14
viewpoint      13

типи геометрії: {'Point': 516, 'Polygon': 86}
```

> **Перевірте себе.** Серед типів геометрії мають бути і `Point`, і `Polygon`. Якщо самі лише полігони — ви відфільтрували зайве: пам'ятники, меморіали та оглядові точки в OSM є **вузлами**, у них немає будівлі.

Якщо кандидатів менше тридцяти

Невелике місто може дати 15–20 об'єктів. Тоді:

1. розширте перелік `tourism` (наприклад, додайте `theatre`, `place_of_worship`);
2. або зменште `N_LARGE` до того, що є;

і **обов'язково зазначте це у звіті** разом із числом.

---

3. Критерій «найкращі»

Ключовий крок роботи. Стовпця «цінність» у даних немає. Його треба сконструювати і захистити.

Сигнали, які є в OSM:

сигнал	що означає
`wikidata` / `wikipedia`	про об'єкт є стаття — хтось сторонній визнав його вартим опису
кількість колонок `name:*`	назву переклали N мовами — перекладають те, що шукають іноземці
`tourism=museum/gallery`	призначений для відвідування
`historic` заповнений	має історичну природу
площа полігона	розмір — **найслабший сигнал**: великим буває і склад

Завдання 3.1

Побудуйте числовий рейтинг. Вимоги:

- використати **щонайменше три** різні сигнали;
- звести різнорідні величини до спільної шкали (підказка: `rank(pct=True)`);
- **пояснити у звіті, чому саме такі ваги**.

Хід виконання. Будується рейтинг цінності. Для кожного кандидата рахується п'ять ознак: чи є стаття у Вікіпедії, скількома мовами перекладено назву, чи призначений об'єкт для відвідування, чи він історичний, і його площа. Кількість мов і площа переводяться у шкалу від 0 до 1 за місцем серед інших, щоб великі числа не задавили ознаки «так/ні». Кожна ознака множиться на свою вагу, і все складається в загальну оцінку. Виводяться 12 найкращих.

In [15]:
```
metric = cand.to_crs(32635)
cand['area_m2'] = np.where(metric.geometry.geom_type.isin(['Polygon', 'Multi
                           metric.area.values, 0.0)

def filled(col):
    return cand[col].notna() if col in cand else pd.Series(False, index=canc

name_cols = [c for c in cand.columns if c.startswith('name:')]
cand['wiki']    = (filled('wikidata') | filled('wikipedia')).astype(int)
cand['n_lang']  = cand[name_cols].notna().sum(axis=1)
cand['visit']   = (cand['tourism'].isin(['museum', 'gallery', 'attraction'])
                   | cand['historic'].isin(['castle', 'fort', 'citywalls', '
cand['is_hist'] = cand['historic'].notna().astype(int)

# спільна шкала 0..1: двійкові ознаки вже 0/1, неперервні — через ранги
cand['p_lang'] = cand['n_lang'].rank(pct=True) * (cand['n_lang'] > 0)
cand['p_area'] = cand['area_m2'].rank(pct=True) * (cand['area_m2'] > 0)

W = {'wiki': 0.35, 'p_lang': 0.30, 'visit': 0.20, 'is_hist': 0.10, 'p_area':
cand['score'] = sum(w * cand[c] for c, w in W.items())

print('кандидатів   :', len(cand))
print('мають wiki   :', int(cand['wiki'].sum()))
print('мають name:* :', int((cand['n_lang'] > 0).sum()), '| медіана мов сере
```

```
      int(cand.loc[cand['n_lang'] > 0, 'n_lang'].median()))
print('для відвідування:', int(cand['visit'].sum()), '| historic:', int(canc
print('ваги:', W)
display(cand.nlargest(12, 'score')[['name', 'historic', 'tourism', 'wiki', '
                                    'area_m2', 'score']].round(3))
```

```
кандидатів   : 602
мають wiki   : 93
мають name:* : 546 | медіана мов серед них: 2
для відвідування: 142 | historic: 283
ваги: {'wiki': 0.35, 'p_lang': 0.3, 'visit': 0.2, 'is_hist': 0.1, 'p_area':
0.05}
```

	name	historic	tourism	wiki	n_lang	area_m2	score
527	Кенаса у Харкові	synagogue	attraction	1	4	296.528	0.989
516	Харківський історичний музей	NaN	museum	1	14	1820.926	0.899
66	Танк Т-34-85	tank	attraction	1	3	0.000	0.892
67	Mark V	tank	attraction	1	3	0.000	0.892
68	122-мм гаубиця зразка 1938 року (М-30)	cannon	attraction	1	3	0.000	0.892
69	76-мм дивізійна гармата зразка 1942 року (ЗІС-3)	cannon	attraction	1	3	0.000	0.892
70	100-мм польова гармата БС-3	cannon	attraction	1	3	0.000	0.892
71	Половецькі баби	yes	attraction	1	3	0.000	0.892
72	Половецькі баби	yes	attraction	1	3	0.000	0.892
73	Половецькі баби	yes	attraction	1	3	0.000	0.892
74	Половецькі баби	yes	attraction	1	3	0.000	0.892
522	Державний музей природи Харківського національ...	NaN	museum	1	4	2109.220	0.891

Завдання 3.2 — обов'язкове порівняння

Побудуйте **другий** рейтинг за найпростішим критерієм — площею будівлі — і порахуйте, скільки об'єктів збігається в перших десяти.

Це головне питання роботи: обидва списки отримані з тих самих даних, тим самим кодом, без жодної помилки. Якщо вони різні — поясніть у звіті, що це означає для слова «найкращі».

Хід виконання. Мій рейтинг порівнюється з найпростішим — за площею. Збігається 0 об'єктів з 10. Для порівняння додано третій рейтинг, лише за Вікіпедією і перекладами: він дає 2 збіги. Також перевіряється стійкість: по черзі прибирається

кожна ознака й рахується, скільки об'єктів десятки лишається. Без деяких ознак лишається лише 2 з 10, тобто склад «найкращих» залежить від обраних ваг.

In [16]:
```
N_SMALL, N_LARGE = 10, 30

def compare(top10_, naive_, label):
    ov = len(set(top10_['name']) & set(naive_['name']))
    print('%s: перетин %d з %d' % (label, ov, N_SMALL))
    return ov

top_score = cand.nlargest(N_LARGE, 'score').reset_index(drop=True)
top10     = top_score.head(N_SMALL)
top_naive = cand.nlargest(N_SMALL, 'area_m2').reset_index(drop=True)
overlap   = compare(top10, top_naive, 'мій критерій vs площа')

# третій погляд: лише «зовнішнє визнання» (wiki + кількість мов), без типу і
top_fame = cand.assign(f=cand['wiki'] + cand['p_lang']).nlargest(N_SMALL, 'f
overlap_fame = compare(top10, top_fame, 'мій критерій vs лише wiki+мови')

display(pd.DataFrame({'за моїм критерієм': list(top10['name']),
                      'за площею': list(top_naive['name']),
                      'площа, м²': top_naive['area_m2'].round(0).astype(int)
                      'тип за площею': (top_naive['historic'].fillna('')
                                        + ' ' + top_naive['tourism'].fillna(

# стійкість до ваг: скільки з десятки лишається, якщо вагу кожного сигналу с
rob = {}
for c in W:
    W2 = {k: (0 if k == c else v) for k, v in W.items()}
    s2 = sum(w * cand[k] for k, w in W2.items())
    rob['без ' + c] = len(set(cand.loc[s2.nlargest(N_SMALL).index, 'name']))
print('скільки з моєї десятки лишається, якщо прибрати один сигнал:', rob)
```

```
мій критерій vs площа: перетин 0 з 10
мій критерій vs лише wiki+мови: перетин 2 з 10
```

	за моїм критерієм	за площею	площа, м²	тип за площею
0	Кенаса у Харкові	Меморіальний комплекс жертв Голокосту "Дробиць...	71109	memorial
1	Харківський історичний музей	Донецьке городище	40574	archaeological_site
2	Танк Т-34-85	Музей історії і техніки Південної залізниці	29578	museum
3	Mark V	Будинок зі шпилем	4406	attraction
4	122-мм гаубиця зразка 1938 року (М-30)	Американська гірка	3424	attraction
5	76-мм дивізійна гармата зразка 1942 року (ЗІС-3)	Будівлі колишнього заводу барона Едварда Ферди...	2788	yes
6	100-мм польова гармата БС-3	Братська та індивідуальні могили радянських во...	2418	memorial
7	Половецькі баби	Автошкола	2412	attraction
8	Половецькі баби	Будівлі колишнього заводу барона Едварда Ферди...	2290	yes
9	Половецькі баби	Старий цирк	2226	yes

```
скільки з моєї десятки лишається, якщо прибрати один сигнал: {'без wiki': 8,
'без p_lang': 7, 'без visit': 2, 'без is_hist': 2, 'без p_area': 7}
```

Завдання 3.3 — дублікати

OSM наповнюють різні люди. Той самий музей трапляється двічі під різними назвами, а вежа буває окремим об'єктом усередині палацу. Знайдіть у `top_score` пари, що стоять ближче **60 метрів** одна до одної, і вирішіть, що з ними робити.

Рішення (лишити / злити / видалити) опишіть у звіті.

Хід виконання. Шукаються пари об'єктів у топ-30, що стоять ближче 60 метрів одна до одної. Спершу функцію перевірено на двох вигаданих точках за 11 м одна від одної. Знайдено 40 таких пар: танки, гармати і «Половецькі баби» біля Історичного музею, а також справжні дублікати.

In [17]:
```
def close_pairs(gdf, limit_m=60):
    p = gdf.to_crs(32635).geometry.representative_point()
    XY = np.c_[p.x.values, p.y.values]
    D = np.sqrt(((XY[:, None, :] - XY[None, :, :]) ** 2).sum(-1))
    np.fill_diagonal(D, np.inf)
    out = [(i, j, D[i, j]) for i in range(len(XY)) for j in range(i + 1, ler
           if D[i, j] < limit_m]
    return sorted(out, key=lambda t: t[2])
```

```
# перевірка, що функція взагалі працює: дві точки за ~11 м мають знайтись
from shapely.geometry import Point
_t = gpd.GeoDataFrame(geometry=[Point(36.2300, 49.99), Point(36.23015, 49.99
assert len(close_pairs(_t)) == 1, 'close_pairs не бачить завідомо близьких т

pairs = close_pairs(top_score)
print('пар ближче 60 м у топ-%d: %d' % (N_LARGE, len(pairs)))
for i, j, d in pairs:
    a, b = top_score.iloc[i], top_score.iloc[j]
    print('  %5.1f м | #%-2d %-45s | #%-2d %s' % (d, i + 1, a['name'][:45],
```

```
пар ближче 60 м у топ-30: 40
    1.5 м | #9  Половецькі баби                                | #10 Половець
кі баби
    1.5 м | #8  Половецькі баби                                | #9  Половець
кі баби
    1.9 м | #10 Половецькі баби                                | #11 Половець
кі баби
    2.9 м | #8  Половецькі баби                                | #10 Половець
кі баби
    3.2 м | #9  Половецькі баби                                | #11 Половець
кі баби
    4.3 м | #5  122-мм гаубиця зразка 1938 року (М-30)         | #6  76-мм ди
візійна гармата зразка 1942 року (ЗІС-3)
    4.4 м | #6  76-мм дивізійна гармата зразка 1942 року (ЗІС | #7  100-мм п
ольова гармата БС-3
    4.4 м | #8  Половецькі баби                                | #11 Половець
кі баби
    8.7 м | #5  122-мм гаубиця зразка 1938 року (М-30)         | #7  100-мм п
ольова гармата БС-3
   16.8 м | #3  Танк Т-34-85                                   | #7  100-мм п
ольова гармата БС-3
   17.9 м | #3  Танк Т-34-85                                   | #4  Mark V
   19.3 м | #15 Дзеркальний струмінь                           | #17 Дзеркаль
ний струмінь
   21.1 м | #3  Танк Т-34-85                                   | #6  76-мм ди
візійна гармата зразка 1942 року (ЗІС-3)
   25.3 м | #3  Танк Т-34-85                                   | #5  122-мм г
аубиця зразка 1938 року (М-30)
   34.4 м | #4  Mark V                                         | #7  100-мм п
ольова гармата БС-3
   38.5 м | #4  Mark V                                         | #6  76-мм ди
візійна гармата зразка 1942 року (ЗІС-3)
   38.9 м | #2  Харківський історичний музей                   | #3  Танк Т-3
4-85
   40.1 м | #2  Харківський історичний музей                   | #7  100-мм п
ольова гармата БС-3
   40.2 м | #4  Mark V                                         | #8  Половець
кі баби
   40.7 м | #2  Харківський історичний музей                   | #6  76-мм ди
візійна гармата зразка 1942 року (ЗІС-3)
   40.9 м | #2  Харківський історичний музей                   | #4  Mark V
   41.5 м | #4  Mark V                                         | #9  Половець
кі баби
   41.6 м | #2  Харківський історичний музей                   | #5  122-мм г
аубиця зразка 1938 року (М-30)
   42.6 м | #4  Mark V                                         | #5  122-мм г
аубиця зразка 1938 року (М-30)
   43.0 м | #4  Mark V                                         | #10 Половець
кі баби
   44.6 м | #4  Mark V                                         | #11 Половець
кі баби
   45.8 м | #2  Харківський історичний музей                   | #8  Половець
кі баби
   47.3 м | #2  Харківський історичний музей                   | #9  Половець
кі баби
   48.1 м | #2  Харківський історичний музей                   | #10 Половець
```

```
кі баби
   48.5 м | #2  Харківський історичний музей                  | #11 Половець
кі баби
   49.9 м | #2  Харківський історичний музей                  | #20 Пам'ятни
к Незалежності "Україна, що летить"
   51.0 м | #14 Будинок зі шпилем                             | #27 Будинок
зі шпилем
   52.8 м | #11 Половецькі баби                               | #20 Пам'ятни
к Незалежності "Україна, що летить"
   54.1 м | #10 Половецькі баби                               | #20 Пам'ятни
к Незалежності "Україна, що летить"
   54.5 м | #8  Половецькі баби                               | #20 Пам'ятни
к Незалежності "Україна, що летить"
   54.7 м | #9  Половецькі баби                               | #20 Пам'ятни
к Незалежності "Україна, що летить"
   55.1 м | #3  Танк Т-34-85                                  | #8  Половець
кі баби
   56.5 м | #3  Танк Т-34-85                                  | #9  Половець
кі баби
   57.9 м | #3  Танк Т-34-85                                  | #10 Половець
кі баби
   59.4 м | #3  Танк Т-34-85                                  | #11 Половець
кі баби
```

Рішення щодо близьких пар. Для пішого маршруту дві точки ближче 60 м — це одна зупинка: турист бачить обидва об'єкти, не роблячи жодного кроку. Якщо лишити обидві, вони займуть два місця з тридцяти (і з десяти) і, швидше за все, сядуть на один вузол графа з нульовою віддаллю. Тому правило: **з пари лишаю об'єкт із вищим рейтингом, другий видаляю, а на звільнене місце піднімається наступний кандидат** (жадібне придушення сусідів, як NMS у комп'ютерному зорі). Видалені об'єкти виводжу, щоб було видно, що пропало.

Хід виконання. Прибираються дублікати. Кандидати переглядаються від найвищого рейтингу до найнижчого. Якщо об'єкт стоїть ближче 60 м до вже взятого, він вважається тією самою зупинкою і відкидається, а його місце займає наступний кандидат. Видалено 12 об'єктів. Після чистки близьких пар не лишилось, і рейтинг ще раз порівнюється з рейтингом за площею: тепер збігів 2 з 10.

In [18]:
```
order = cand.sort_values('score', ascending=False)
P = order.to_crs(32635).geometry.representative_point()
kept, dropped = [], []
for idx, p in zip(order.index, P):
    near = [k for k in kept if P[k].distance(p) < 60]
    if near:
        dropped.append((cand.at[idx, 'name'], cand.at[near[0], 'name'], P[ne
        continue
    kept.append(idx)
    if len(kept) == N_LARGE:
        break

top_score = cand.loc[kept].reset_index(drop=True)
top10 = top_score.head(N_SMALL)
```

```
print('видалено як дублікат / частину сусіднього об’єкта: %d' % len(dropped))
for a, b, d in dropped:
    print('  – %-45s (за %.0f м від «%s»)' % (a[:45], d, b))
print()
print('пар ближче 60 м після чистки:', len(close_pairs(top_score)))
overlap = compare(top10, top_naive, 'мій критерій (після чистки) vs площа')
print()
print('ТОП-10:'); print(top10[['name', 'historic', 'tourism', 'wiki', 'n_lar
                       .round(3).to_string())
```

видалено як дублікат / частину сусіднього об’єкта: 12
  – 122-мм гаубиця зразка 1938 року (М-30)   (за 42 м від «Харківський історичний музей»)
  – Mark V   (за 41 м від «Харківський історичний музей»)
  – 100-мм польова гармата БС-3   (за 40 м від «Харківський історичний музей»)
  – Половецькі баби   (за 47 м від «Харківський історичний музей»)
  – Половецькі баби   (за 48 м від «Харківський історичний музей»)
  – Половецькі баби   (за 48 м від «Харківський історичний музей»)
  – Танк Т-34-85   (за 39 м від «Харківський історичний музей»)
  – Половецькі баби   (за 46 м від «Харківський історичний музей»)
  – 76-мм дивізійна гармата зразка 1942 року (ЗІС (за 41 м від «Харківський історичний музей»)
  – Дзеркальний струмінь   (за 19 м від «Дзеркальний струмінь»)
  – Пам'ятник Незалежності "Україна, що летить"   (за 50 м від «Харківський історичний музей»)
  – Будинок зі шпилем   (за 51 м від «Будинок зі шпилем»)

пар ближче 60 м після чистки: 0
мій критерій (після чистки) vs площа: перетин 2 з 10

ТОП-10:

	name	historic	tourism	wiki	n_lang	score
0	Кенаса у Харкові	synagogue	attraction	1	4	0.989
1	Харківський історичний музей	NaN	museum	1	14	0.899
2	Державний музей природи Харківського національного університету ім. В. Н. Каразіна	NaN	museum	1	4	0.891
3	Музей-планетарій "Харків космічний"	NaN	museum	1	4	0.890
4	Будинок зі шпилем	NaN	attraction	1	3	0.842
5	Дзеркальний струмінь	NaN	attraction	1	3	0.840
6	Науково-просвітницький музей сексуальних культур світу	NaN	museum	1	3	0.839
7	Меморіальний комплекс жертв Голокосту "Дробицький яр"	memorial	NaN	1	8	0.798
8	Будинок Шапара	building	NaN	1	7	0.796
9	Пам'ятник Воїну-визволителю	monument	NaN	1	12	0.794

4. Старт: вокзал, і прив'язка точок до графа

Хід виконання. Переглядаються станції в шарі, щоб знайти залізничний вокзал. У шарі 88 станцій, і більшість із них — станції метро. Тому вокзал треба вибирати за окремим правилом.

In [19]:
```
print('станції у шарі:', len(stations))
print(stations[['name', 'railway', 'building']].dropna(how='all').head(12).t
```

```
станції у шарі: 88
                      name  railway building
0              Новоселівка     halt      NaN
1         Історичний музей  station      NaN
2              Університет  station      NaN
3                Вокзальна  station      NaN
4       Майдан Конституції  station      NaN
5                   Левада  station      NaN
6                Спортивна  station      NaN
7                Заводська  station      NaN
8                Турбоатом  station      NaN
9             Палац Спорту  station      NaN
10               Армійська  station      NaN
11  Ім. О.С. Масельського  station      NaN
```

Хід виконання. Обирається стартова точка — головний вокзал. Шукається станція зі словом «Пасажирський» у назві, а серед знайдених береться найбільша будівля, бо люди заходять саме в будівлю вокзалу. Знайдено вокзал Харків-Пасажирський.

In [20]:
```
st = stations.copy()
st['name'] = st['name'].astype('string')
st['area'] = st.to_crs(32635).area
city_word = CITY.split(',')[0].strip()

def pick_station(st):
    named = st[st['name'].notna()]
    # 1) головний пасажирський вокзал за назвою
    for pat in ['Пасажирськ', 'Головний', 'вокзал', city_word]:
        hit = named[named['name'].str.contains(pat, case=False, na=False)
                    & (named['railway'].isin(['station']) | named['building'
        if len(hit):
            # будівля вокзалу краща за точку станції: вона там, куди заходят
            return hit.sort_values('area', ascending=False).head(1), pat
    # 2) найбільша будівля вокзалу
    b = st[st['building'].eq('train_station')]
    return b.sort_values('area', ascending=False).head(1), 'найбільша train_

station, how = pick_station(st)
print('правило:', how)
print('старт:', station['name'].iloc[0], '|', station.geometry.iloc[0].geom_
      '| %.5f, %.5f' % (station.geometry.iloc[0].representative_point().y,
                        station.geometry.iloc[0].representative_point().x))
```

```
правило: Пасажирськ
старт: Вокзал Харків-Пасажирський | Polygon | 49.98948, 36.20440
```

Хід виконання. З пішохідної мережі будується граф: перехрестя стають вузлами, а відрізки доріг — ребрами з довжиною в метрах. Якщо між двома перехрестями кілька доріг, лишається найкоротша. Також описується, як прив'язати кожну пам'ятку до найближчого перехрестя, бо ходити можна лише дорогами. Відстань рахується в метрах, а не в градусах.

In [21]:
```
G = nx.DiGraph()
for t in nodes.itertuples():
    G.add_node(t.osmid, x=t.x, y=t.y)
for t in edges.itertuples():
    L = float(t.length)
    if G.has_edge(t.u, t.v):
        G[t.u][t.v]['length'] = min(G[t.u][t.v]['length'], L)
    else:
        G.add_edge(t.u, t.v, length=L)
print('граф: %d вузлів, %d ребер' % (G.number_of_nodes(), G.number_of_edges(

NODES_M = nodes.to_crs(32635)
NODE_XY = np.c_[NODES_M.geometry.x.values, NODES_M.geometry.y.values]
NODE_ID = nodes['osmid'].values

def snap(points_gdf):
    """Найближчий вузол графа до кожної точки. Рахуємо в метричній CRS."""
    pmet = points_gdf.to_crs(32635)
    ids, dist = [], []
    for p in pmet.geometry:
        d2 = (NODE_XY[:, 0] - p.x) ** 2 + (NODE_XY[:, 1] - p.y) ** 2
        k = int(np.argmin(d2))
        ids.append(NODE_ID[k]); dist.append(float(np.sqrt(d2[k])))
    return np.array(ids), np.array(dist)

def make_points(top):
    names = ['СТАРТ: ' + str(station['name'].iloc[0])] + list(top['name'])
    geoms = [station.geometry.iloc[0].representative_point()] + \
            [g.representative_point() for g in top.geometry]
    return gpd.GeoDataFrame({'name': names}, geometry=geoms, crs='EPSG:4326'
```

```
граф: 65588 вузлів, 187308 ребер
```

Завдання 4.1 — обов'язкова перевірка

Напишіть `check_snap`, яка повідомляє:

- скільки точок і скільки **різних** вузлів вони зайняли;
- медіану й максимум відстані «точка → вузол»;
- якщо вузлів менше, ніж точок — **назви точок, що злиплись**.

Контрольна ознака: на пішохідній мережі різних вузлів має бути **стільки ж, скільки точок**. Якщо менше — ви взяли автомобільну мережу або місто надто мале; зафіксуйте це у звіті, бо далі такі пари отримають нульову віддаль.

Хід виконання. Описується перевірка прив'язки: скільки точок і скільки різних перехресть вони зайняли, яка середня і найбільша відстань до перехрестя. Якщо дві пам'ятки сіли на одне перехрестя, виводяться їхні назви, бо відстань між ними стала б нульовою.

In [22]:
```
def check_snap(points, ids, dist, label):
    n_pts, n_nodes = len(ids), len(set(ids))
    print('[%s] точок: %d | різних вузлів: %d | точка→вузол: медіана %.1f м,
          % (label, n_pts, n_nodes, np.median(dist), dist.max(),
             points['name'].iloc[int(np.argmax(dist))]))
    if n_nodes < n_pts:
        s = pd.Series(points['name'].values).groupby(ids).agg(list)
        for node, names in s[s.map(len) > 1].items():
            print('   ЗЛИПЛИСЬ на вузлі %s: %s' % (node, ' + '.join(names)))
    else:
        print('   злиплих точок немає — кожна пам’ятка має власний вузол')
    return n_nodes == n_pts
```

---

5. Матриця віддалей

Дві матриці: **пряма лінія** і **дорогою**. Їхнє відношення — коефіцієнт обходу — показує, наскільки пряма лінія бреше.

Завдання 5.1

Хід виконання. Описуються три розрахунки. Перший — таблиця відстаней дорогою між усіма точками: для кожної точки шукаються найкоротші шляхи до решти. Другий — таблиця відстаней по прямій. Третій — коефіцієнт обходу, тобто у скільки разів дорога довша за пряму.

In [23]:
```
def road_matrix(ids):
    """Матриця найкоротших віддалей графом між заданими вузлами."""
    k = len(ids)
    D = np.full((k, k), np.inf)
    for i, s in enumerate(ids):
        lengths = nx.single_source_dijkstra_path_length(G, s, weight='length
        for j, t in enumerate(ids):
            D[i, j] = lengths.get(t, np.inf)
    np.fill_diagonal(D, 0.0)
    return D

def straight_matrix(points):
    p = points.to_crs(32635).geometry
    XY = np.c_[p.x.values, p.y.values]
    return np.sqrt(((XY[:, None, :] - XY[None, :, :]) ** 2).sum(-1))

def detour_factor(D, E):
```

```
    off = ~np.eye(len(D), dtype=bool) & np.isfinite(D) & (E > 0)
    return float(np.median(D[off] / E[off]))
```

Контрольні ознаки — звіртесь, не підганяючи:

- коефіцієнт обходу для пішохідної мережі у місті зазвичай **1,2–1,6**;
- якщо у вас рівно **1,0** — ви порівнюєте матрицю саму із собою;
- якщо **більше 2** — найімовірніше взято автомобільну мережу з односторонніми вулицями, або місто має річку/залізницю, яку нема де перетнути. Друге — цікавий результат, перше — помилка. Розберіться, що саме у вас, і напишіть це.

Якщо серед віддалей трапились `inf` — частина графа не зв'язана з рештою. Не замінюйте `inf` великим числом мовчки: спершу з'ясуйте, які це точки.

Хід виконання. Розрахунки запускаються для малої задачі: вокзал і 10 пам'яток. Прив'язка правильна: 11 точок зайняли 11 різних перехресть. Таблиця відстаней порахувалась менш ніж за секунду. Недосяжних пар немає, а шлях туди й назад однаковий, бо для пішоходів немає одностороннього руху. Коефіцієнт обходу — 1,15.

In [24]:
```
pts10 = make_points(top10)
ids10, dist10 = snap(pts10)
check_snap(pts10, ids10, dist10, 'мала задача')

t0 = time.time(); D10 = road_matrix(ids10); print('матриця за %.3f с' % (tim
E10 = straight_matrix(pts10)
print('недосяжних пар    :', int(np.isinf(D10).sum()))
print('асиметрія         : %.0f м' % float(np.max(np.abs(D10 - D10.T))))
print('коефіцієнт обходу : %.2f' % detour_factor(D10, E10))
```

```
[мала задача] точок: 11 | різних вузлів: 11 | точка→вузол: медіана 15.0 м, м
акс 498.5 м (Меморіальний комплекс жертв Голокосту "Дробицький яр")
   злиплих точок немає — кожна пам’ятка має власний вузол
матриця за 0.806 с
недосяжних пар    : 0
асиметрія         : 0 м
коефіцієнт обходу : 1.15
```

Хід виконання. Коефіцієнт обходу розглядається для кожної пари окремо, щоб його пояснити. Він коливається від 1,02 до 1,79. Найбільший обхід — від Історичного музею до планетарію: 370 м по прямій і 661 м дорогою.

In [25]:
```
# Де пряма лінія бреше найбільше — щоб пояснити коефіцієнт, а не лише назват
R10 = np.where(E10 > 0, D10 / np.where(E10 > 0, E10, 1), np.nan)
np.fill_diagonal(R10, np.nan)
i, j = np.unravel_index(np.nanargmax(R10), R10.shape)
print('коефіцієнт обходу по парах: мін %.2f | медіана %.2f | макс %.2f'
      % (np.nanmin(R10), np.nanmedian(R10), np.nanmax(R10)))
print('найбільший обхід: %s -> %s: пряма %.0f м, дорогою %.0f м'
      % (pts10['name'][i], pts10['name'][j], E10[i, j], D10[i, j]))
```

```
коефіцієнт обходу по парах: мін 1.02 | медіана 1.15 | макс 1.79
найбільший обхід: Харківський історичний музей -> Музей-планетарій "Харків к
осмічний": пряма 370 м, дорогою 661 м
```

> **Асиметрія.**
> D_{ij}
> $neq D_{ji}$
> — не помилка: односторонній рух робить шлях туди і назад різним. На пішохідній мережі асиметрія мала, на автомобільній — помітна.
> Подивіться на своє число і поясніть його у звіті.

---

6. Задача комівояжера

метод	що робить	складність
найближчий сусід	щоразу йде в найближчу невідвідану точку	$O(n^2)$
2-opt	розвертає ділянки готового маршруту, поки коротшає	$O(n^2)$ за прохід
Held–Karp	динамічне програмування за підмножинами — **точний**	$O(2^n n^2)$ часу, $O(2^n n)$ пам'яті

Held–Karp дано готовим: він не є метою роботи, але без нього ви не дізнаєтесь, **наскільки** помиляються перші два.

Завдання 6.1 — реалізуйте перші два методи

Хід виконання. Описуються два наближені методи пошуку маршруту. Найближчий сусід щоразу йде в найближчу невідвідану точку. 2-opt бере готовий маршрут і перевертає його ділянки, поки маршрут коротшає, — так розплутуються петлі. Також є функція, що рахує довжину маршруту.

In [26]:
```python
def tour_length(D, tour):
    return float(sum(D[tour[i], tour[i + 1]] for i in range(len(tour) - 1)))

def nearest_neighbour(D):
    """Маршрут 0 -> ... -> 0. Щоразу йдемо в найближчу невідвідану точку."""
    n = len(D)
    tour, left = [0], set(range(1, n))
    while left:
        cur = tour[-1]
        nxt = min(left, key=lambda j: (D[cur, j], j))   # j — для відтворюва
        tour.append(nxt); left.remove(nxt)
    return tour + [0]

def two_opt(D, tour):
    """Беремо маршрут і розвертаємо ділянки, поки довжина зменшується."""
```

```
    best = list(tour)
    best_len = tour_length(D, best)
    improved = True
    while improved:
        improved = False
        for i in range(1, len(best) - 2):
            for k in range(i + 1, len(best) - 1):
                cand_t = best[:i] + best[i:k + 1][::-1] + best[k + 1:]
                # повна довжина, а не дельта двох ребер: матриця може бути
                # несиметричною, і розвернута ділянка тоді теж змінює довжин
                L = tour_length(D, cand_t)
                if L < best_len - 1e-9:
                    best, best_len, improved = cand_t, L, True
    return best
```

Хід виконання. Описується точний метод Held–Karp, який дано готовим. Він перебирає всі можливі набори вже відвіданих точок і для кожного запам'ятовує найкоротший шлях. Так гарантовано знаходиться найкращий маршрут, але пам'яті для цього потрібно дуже багато.

In [27]:
```
def held_karp(D):
    """Точний розв'язок. Вузол 0 — старт і фініш. ДАНО, писати не треба."""
    k = len(D); N = k - 1; INF = float('inf')
    dp = [[INF] * N for _ in range(1 << N)]
    par = [[-1] * N for _ in range(1 << N)]
    for j in range(N):
        dp[1 << j][j] = D[0, j + 1]
    for S in range(1 << N):
        for j in range(N):
            cur = dp[S][j]
            if cur == INF or not (S >> j) & 1:
                continue
            for t in range(N):
                if (S >> t) & 1:
                    continue
                S2 = S | (1 << t); v = cur + D[j + 1, t + 1]
                if v < dp[S2][t]:
                    dp[S2][t] = v; par[S2][t] = j
    full = (1 << N) - 1
    bj = min(range(N), key=lambda j: dp[full][j] + D[j + 1, 0])
    best = dp[full][bj] + D[bj + 1, 0]
    path = []; S = full; j = bj
    while j != -1:
        path.append(j + 1); pj = par[S][j]; S ^= (1 << j); j = pj
    return [0] + path[::-1] + [0], float(best)
```

Самоперевірка

Ці твердження мають виконатись для **будь-якого** міста. Якщо хоч одне впало — шукайте помилку в коді, а не в даних.

Хід виконання. Усі три методи запускаються на 10 точках, і їхні результати перевіряються. Маршрут має починатися й закінчуватися на вокзалі, а кожна точка має бути в ньому рівно один раз. 2-opt не може бути гіршим за найближчого сусіда, а точний метод — гіршим за 2-opt. Усі перевірки пройдено.

In [28]:
```
def check_tour(tour, k, label):
    assert tour[0] == 0 and tour[-1] == 0, '%s: маршрут має починатись і зак
    assert sorted(tour[:-1]) == list(range(k)), '%s: кожна точка рівно один
    print('%s: структура маршруту правильна (%d точок)' % (label, k))

nn10 = nearest_neighbour(D10)
op10 = two_opt(D10, nn10)
ex10, L_opt = held_karp(D10)

for t, lbl in [(nn10, 'найближчий сусід'), (op10, '2-opt'), (ex10, 'Held-Kar
    check_tour(t, len(ids10), lbl)

assert tour_length(D10, op10) <= tour_length(D10, nn10) + 1e-6, '2-opt не мо
assert L_opt <= tour_length(D10, op10) + 1e-6, 'точний розв\u2019язок не мож
print()
print('усі перевірки пройдено')
```

```
найближчий сусід: структура маршруту правильна (11 точок)
2-opt: структура маршруту правильна (11 точок)
Held-Karp: структура маршруту правильна (11 точок)

усі перевірки пройдено
```

Хід виконання. Порівнюються довжини маршрутів трьох методів. Найближчий сусід дає 52,4 км, тобто на 4,3 % гірше за найкращий маршрут. 2-opt дає 50,36 км (гірше на 0,18 %), а точний метод — 50,27 км. Виводиться найкращий порядок обходу.

In [29]:
```
rows = []
for name, tour in [('найближчий сусід', nn10), ('2-opt', op10), ('Held-Karp
    L = tour_length(D10, tour)
    rows.append({'метод': name, 'довжина, м': round(L),
                 'гірше оптимуму, %': round(100 * (L / L_opt - 1), 2)})
display(pd.DataFrame(rows).set_index('метод'))

print()
print('оптимальний порядок:')
for step, i in enumerate(ex10):
    print('  %2d. %s' % (step, pts10['name'].iloc[i]))
print()
print('довжина: %.2f км' % (L_opt / 1000))
```

метод	довжина, м	гірше оптимуму, %
найближчий сусід	52431	4.30
2-opt	50362	0.18
Held-Karp (точний)	50271	0.00

```
оптимальний порядок:
   0. СТАРТ: Вокзал Харків-Пасажирський
   1. Будинок Шапара
   2. Будинок зі шпилем
   3. Кенаса у Харкові
   4. Меморіальний комплекс жертв Голокосту "Дробицький яр"
   5. Науково-просвітницький музей сексуальних культур світу
   6. Пам'ятник Воїну-визволителю
   7. Державний музей природи Харківського національного університету ім. В.
Н. Каразіна
   8. Дзеркальний струмінь
   9. Музей-планетарій "Харків космічний"
  10. Харківський історичний музей
  11. СТАРТ: Вокзал Харків-Пасажирський

довжина: 50.27 км
```

Завдання 6.2 — велика задача

Спершу оцініть, чи можливий тут точний розв'язок. Порахуйте для $n = 11, 16, 21, 26, 31$ розмір таблиці Held–Karp ($2^{n-1} cdot(n-1)$ клітин) і скільки це гігабайтів при 8 байтах на клітину.

Хід виконання. Оцінюється, чи можна точно розв'язати задачу на 30 точок. Для різної кількості точок рахується розмір таблиці точного методу і скільки пам'яті вона займе. На 31 точці потрібно 258 ГБ, а на ноутбуці 17 ГБ, тобто в 15 разів менше. Отже, точний розв'язок неможливий через пам'ять.

In [30]:
```
rows = []
for n in [11, 16, 21, 26, 31]:
    cells_ = 2 ** (n - 1) * (n - 1)
    rows.append({'точок n': n, 'клітин': '%.1e' % cells_,
                 'пам’ять dp, ГБ': round(cells_ * 8 / 1e9, 3),
                 'dp + par, ГБ': round(cells_ * 16 / 1e9, 3),
                 'операцій ~2ⁿ·n²': '%.1e' % (2 ** (n - 1) * (n - 1) ** 2)})
hk = pd.DataFrame(rows).set_index('точок n')
display(hk)

import platform, subprocess
try:
    ram = int(subprocess.check_output(['sysctl', '-n', 'hw.memsize'])) / 1e9
```

```
except Exception:
    ram = float('nan')
print('оперативна пам’ять цього комп’ютера: %.0f ГБ' % ram)
print('потрібно для n=31: %.0f ГБ лише під dp (у %.0f разів більше)'
      % (hk.loc[31, 'пам’ять dp, ГБ'], hk.loc[31, 'пам’ять dp, ГБ'] / ram))
print('A held_karp із цього ноутбука тримає dp і par як списки Python-об’єкт
print('~8 байт на посилання + сам float/int, тобто реально в 3–4 рази більше
```

точок n	клітин	пам’ять dp, ГБ	dp + par, ГБ	операцій ~2ⁿ·n²
11	1.0e+04	0.000	0.000	1.0e+05
16	4.9e+05	0.004	0.008	7.4e+06
21	2.1e+07	0.168	0.336	4.2e+08
26	8.4e+08	6.711	13.422	2.1e+10
31	3.2e+10	257.698	515.396	9.7e+11

```
оперативна пам’ять цього комп’ютера: 17 ГБ
потрібно для n=31: 258 ГБ лише під dp (у 15 разів більше)
A held_karp із цього ноутбука тримає dp і par як списки Python-об’єктів:
~8 байт на посилання + сам float/int, тобто реально в 3–4 рази більше.
```

Подивіться на останній рядок і напишіть у звіті, **чому саме** точний розв'язок на 30 точках недоступний. Слово «довго» відповіддю не є.

Хід виконання. Розв'язується велика задача: вокзал і 30 пам'яток. Прив'язка знову правильна (31 точка — 31 перехрестя), коефіцієнт обходу — 1,19. Використовуються лише наближені методи: найближчий сусід дає 75,8 км, а 2-opt — 67,6 км, тобто на 10,7 % коротше. Це база для порівняння в лабораторній роботі № 8.

In [31]:
```
pts30 = make_points(top_score)
ids30, dist30 = snap(pts30)
check_snap(pts30, ids30, dist30, 'велика задача')

D30 = road_matrix(ids30)
E30 = straight_matrix(pts30)
print('коефіцієнт обходу: %.2f' % detour_factor(D30, E30))

nn30 = nearest_neighbour(D30)
t0 = time.time(); op30 = two_opt(D30, nn30); t_op30 = time.time() - t0
check_tour(op30, len(ids30), '2-opt на 30 точках')

L_nn30, L_op30 = tour_length(D30, nn30), tour_length(D30, op30)
print()
print('найближчий сусід : %8.0f м' % L_nn30)
print('2-opt            : %8.0f м  за %.3f с  (коротше на %.1f%%)'
      % (L_op30, t_op30, 100 * (1 - L_op30 / L_nn30)))
print()
print('ЦЕ — база порівняння для лабораторної роботи № 8.')
```

```
[велика задача] точок: 31 | різних вузлів: 31 | точка→вузол: медіана 15.0 м,
макс 498.5 м (Меморіальний комплекс жертв Голокосту "Дробицький яр")
   злиплих точок немає — кожна пам’ятка має власний вузол
коефіцієнт обходу: 1.19
2-opt на 30 точках: структура маршруту правильна (31 точок)

найближчий сусід :    75768 м
2-opt            :    67648 м  за 0.005 с  (коротше на 10.7%)

ЦЕ — база порівняння для лабораторної роботи № 8.
```

---

7. Просторова індексація: H3

H3 — ієрархічна сітка шестикутників. Рівень 9 — приблизно 200 м по ребру.

📖 https://h3geo.org/docs/core-library/restable/

Завдання 7.1

Розкладіть **усіх** кандидатів по комірках H3 рівня 9 і побудуйте картограму щільності. На тій самій карті покажіть межу міста і точки вашого маршруту.

Хід виконання. Місто ділиться на шестикутники H3 зі стороною близько 200 м, і в кожному рахується кількість пам'яток. Будуються дві карти: щільність по всьому місту і збільшений центр із точками маршруту. Половина з 602 пам'яток зосереджена лише в 33 шестикутниках із 228, а 10 точок маршруту лежать у 10 різних шестикутниках.

In [32]:
```
import h3
from shapely.geometry import Polygon

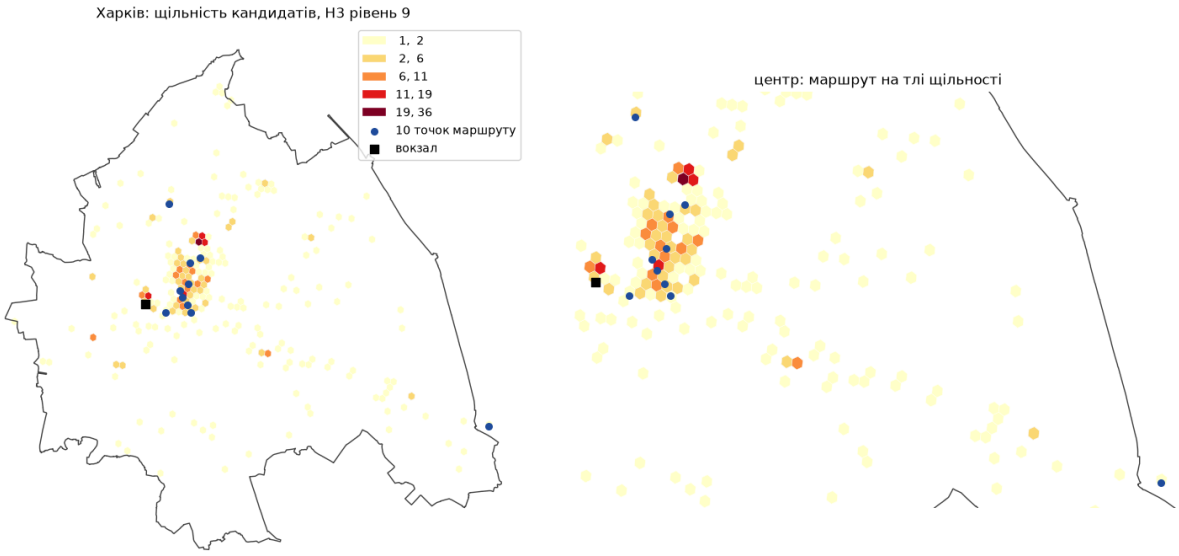
H3_RES = 9
print('середнє ребро комірки: %.0f м' % h3.average_hexagon_edge_length(H3_RE

rp = cand.geometry.representative_point()
cand['h3'] = [h3.latlng_to_cell(p.y, p.x, H3_RES) for p in rp]
dens = cand['h3'].value_counts()
hexes = gpd.GeoDataFrame(
    {'h3': dens.index, 'n': dens.values},
    geometry=[Polygon([(lng, lat) for lat, lng in h3.cell_to_boundary(c)]) f
    crs='EPSG:4326')

cum = dens.cumsum() / dens.sum()
half_cells = int((cum < 0.5).sum() + 1)
print('кандидатів: %d у %d комірках' % (len(cand), len(dens)))
print('половина всіх пам’яток — у %d комірках (%.1f%% зайнятих комірок)'
      % (half_cells, 100 * half_cells / len(dens)))
print('найщільніша комірка: %d об’єктів' % dens.iloc[0])
top10_cells = {h3.latlng_to_cell(p.y, p.x, H3_RES) for p in top10.geometry.r
print('десятка маршруту лежить у %d різних комірках' % len(top10_cells))
```

```
fig, ax = plt.subplots(1, 2, figsize=(15, 7))
for a, zoom in zip(ax, [False, True]):
    hexes.plot(ax=a, column='n', cmap='YlOrRd', legend=not zoom, scheme='Nat
               k=5, edgecolor='white', linewidth=0.3,
               legend_kwds={'title': 'пам’яток у комірці', 'loc': 'lower lef
    boundary.boundary.plot(ax=a, color='#444', linewidth=1)
    pp = make_points(top10)
    pp.iloc[1:].plot(ax=a, color='#1f4e9c', markersize=30, zorder=5, label='
    pp.iloc[:1].plot(ax=a, color='black', marker='s', markersize=50, zorder=
    if zoom:
        xs, ys = pp.geometry.x, pp.geometry.y
        a.set_xlim(xs.min() - 0.01, xs.max() + 0.01); a.set_ylim(ys.min() -
        a.set_title('центр: маршрут на тлі щільності')
    else:
        a.set_title('%s: щільність кандидатів, H3 рівень %d' % (city_word, H
        a.legend(loc='upper right')
    a.set_axis_off()
plt.tight_layout(); plt.show()
```

```
середнє ребро комірки: 201 м
кандидатів: 602 у 228 комірках
половина всіх пам’яток — у 33 комірках (14.5% зайнятих комірок)
найщільніша комірка: 36 об’єктів
десятка маршруту лежить у 10 різних комірках
```



> **Пастка.** `h3.cell_to_boundary` повертає пари **(lat, lng)**, а `shapely.Polygon` чекає **(x, y)**, тобто (lng, lat). Переплутаєте — місто опиниться в Індійському океані. Це видно одразу, тому дивіться на карту.

Питання до звіту

У скількох комірках зосереджено половину всіх пам'яток вашого міста? Що це говорить про форму маршруту?

---

8. Просторове поєднання

Завдання 8.1

Для кожної точки маршруту знайдіть найближчий заклад із шару `amenities` (`sjoin_nearest`) і виведіть таблицю з відстанню в метрах.

Нагадування: `sjoin_nearest` рахує в одиницях CRS. У градусах результат буде безглуздим — перепроєктуйте обидва шари в метричну систему.

Хід виконання. Для кожної точки маршруту шукається найближчий заклад: кафе, ресторан, туалет чи питна вода. Перед цим обидва шари переводяться в метри. Зазвичай заклад є в межах 58 м, а найдалі він біля «Дробицького яру» (557 м). Окремо рахується відстань до питної води й туалетів, бо для пішого маршруту вони найважливіші.

In [33]:
```
route_pts = make_points(top10)
am = amenities.copy()
am['geometry'] = am.geometry.representative_point()      # полігони кафе ->
am_m = am[['amenity', 'name', 'geometry']].to_crs(32635)
rp_m = route_pts.to_crs(32635)

near = gpd.sjoin_nearest(rp_m, am_m, how='left', distance_col='відстань_м',
                         lsuffix='точка', rsuffix='заклад')
near = near[~near.index.duplicated()]                     # рівновіддалені -
tbl = pd.DataFrame({'точка маршруту': near['name_точка'].values,
                    'найближчий заклад': near['name_заклад'].fillna('(без на
                    'тип': near['amenity'].values,
                    'відстань, м': near['відстань_м'].round(0).astype(int).v
display(tbl)
print('медіана до найближчого закладу: %.0f м, максимум %.0f м'
      % (tbl['відстань, м'].median(), tbl['відстань, м'].max()))

# окремо — питна вода й туалети: для пішого маршруту важливіші за кафе
for kind in ['drinking_water', 'toilets']:
    sub = am_m[am_m['amenity'] == kind]
    if len(sub):
        d = gpd.sjoin_nearest(rp_m, sub, distance_col='d')['d']
        d = d[~d.index.duplicated()]
        print('%-15s: медіана %.0f м, максимум %.0f м (об’єктів у місті: %d)
              % (kind, d.median(), d.max(), len(sub)))
```

	точка маршруту	найближчий заклад	тип	відстань, м
0	СТАРТ: Вокзал Харків-Пасажирський	(без назви)	toilets	30
1	Кенаса у Харкові	merquel coffe with love	cafe	58
2	Харківський історичний музей	(без назви)	toilets	55
3	Державний музей природи Харківського національ...	Twins Харків	cafe	118
4	Музей-планетарій "Харків космічний"	Райський куточок	restaurant	29
5	Будинок зі шпилем	IL Monte	cafe	13
6	Дзеркальний струмінь	Переїдальня	restaurant	95
7	Науково-просвітницький музей сексуальних культ...	True Coffee	cafe	61
8	Меморіальний комплекс жертв Голокосту "Дробиць...	Боярський Дворик	restaurant	557
9	Будинок Шапара	Prohibition Pizzeria	restaurant	245
10	Пам'ятник Воїну-визволителю	(без назви)	toilets	42

```
медіана до найближчого закладу: 58 м, максимум 557 м
drinking_water : медіана 329 м, максимум 2194 м (об’єктів у місті: 259)
toilets        : медіана 273 м, максимум 3739 м (об’єктів у місті: 118)
```

---

9. Маршрут на карті

Лінія маршруту — це **геометрія ребер графа**, а не відрізки між пам'ятками.

Завдання 9.1

Побудуйте карту, на якій **обидві** лінії: справжній маршрут по ребрах і прямі відрізки між точками. Різниця має бути видно оком.

Хід виконання. Будується інтерактивна карта маршруту. Справжній маршрут вулицями відновлюється з графа відрізок за відрізком і показується червоною лінією, а поверх неї пунктиром — прямі між точками. Зупинки пронумеровано, а маршрут на 30 точок можна ввімкнути окремим шаром. Карта зберігається у файл `lr5_маршрут.html`. Дорогою маршрут має 50,27 км, а по прямій 46,06 км, тобто пряма недооцінює його на 8 %.

In [34]:
```
node_xy = nodes.set_index('osmid')[['x', 'y']]

def route_coords(tour, ids):
```

```
    """Координати маршруту по ребрах графа у форматі (lat, lon) для folium."
    coords = []
    for a, b in zip(tour[:-1], tour[1:]):
        try:
            path = nx.shortest_path(G, ids[a], ids[b], weight='length')
        except nx.NetworkXNoPath:
            path = [ids[a], ids[b]]
        for nd in path:
            coords.append((float(node_xy.loc[nd, 'y']), float(node_xy.loc[nd
    return coords

road10 = route_coords(ex10, ids10)
straight10 = [(pts10.geometry.y.iloc[i], pts10.geometry.x.iloc[i]) for i in

center = [float(np.mean([c[0] for c in road10])), float(np.mean([c[1] for c
fmap = folium.Map(location=center, zoom_start=14, tiles=None, control_scale=
folium.TileLayer(tiles=BASEMAP.build_url(), attr=BASEMAP.html_attribution +
                 name='Esri World Gray Canvas').add_to(fmap)

g_road = folium.FeatureGroup(name='маршрут по ребрах графа (%.2f км)' % (L_o
folium.PolyLine(road10, color='#c0392b', weight=5, opacity=0.85,
                tooltip='по вулицях: %.0f м' % L_opt).add_to(g_road)

L_straight10 = tour_length(E10, ex10)
g_str = folium.FeatureGroup(name='прямі відрізки (%.2f км)' % (L_straight10
folium.PolyLine(straight10, color='#2c3e50', weight=2, dash_array='6 6',
                tooltip='напряму: %.0f м' % L_straight10).add_to(g_str)

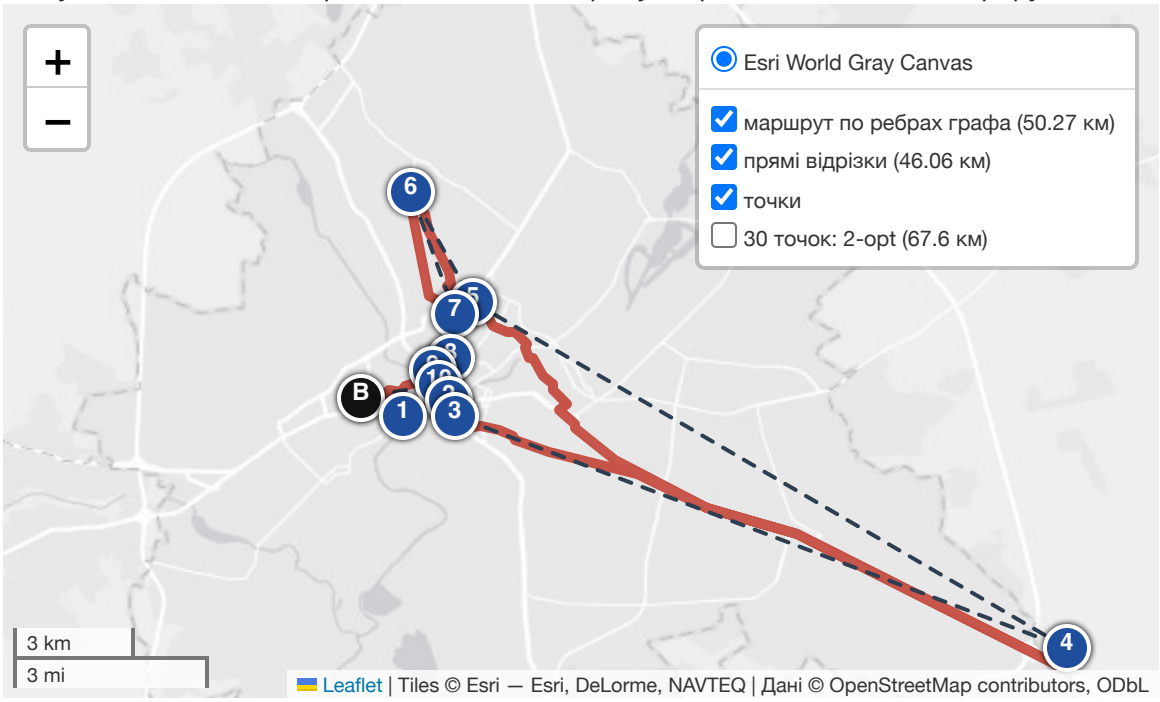
g_pts = folium.FeatureGroup(name='точки').add_to(fmap)
for step, i in enumerate(ex10[:-1]):
    lat, lon = pts10.geometry.y.iloc[i], pts10.geometry.x.iloc[i]
    label = 'B' if step == 0 else str(step)
    bg = '#111' if step == 0 else '#1f4e9c'
    folium.Marker(
        [lat, lon], tooltip='%s. %s' % (label, pts10['name'].iloc[i]),
        icon=folium.DivIcon(html=(
            '<div style="background:%s;color:#fff;border-radius:50%%;width:2
            'line-height:24px;text-align:center;font:bold 12px sans-serif;'
            'border:2px solid #fff;box-shadow:0 0 3px #000">%s</div>' % (bg,
            icon_size=(24, 24), icon_anchor=(12, 12))).add_to(g_pts)

# велика задача — окремим шаром, вимкненим за замовчуванням
g30 = folium.FeatureGroup(name='30 точок: 2-opt (%.1f км)' % (L_op30 / 1000)
folium.PolyLine(route_coords(op30, ids30), color='#8e44ad', weight=3, opacit
for i in range(1, len(pts30)):
    folium.CircleMarker([pts30.geometry.y.iloc[i], pts30.geometry.x.iloc[i]]
                        color='#8e44ad', fill=True, tooltip=pts30['name'].il

folium.LayerControl(collapsed=False).add_to(fmap)
fmap.fit_bounds([[min(c[0] for c in road10), min(c[1] for c in road10)],
                 [max(c[0] for c in road10), max(c[1] for c in road10)]])
fmap.save('lr5_маршрут.html')
print('збережено lr5_маршрут.html')
print('по вулицях %.2f км проти %.2f км напряму: пряма недооцінює маршрут на
      % (L_opt / 1000, L_straight10 / 1000, 100 * (1 - L_straight10 / L_opt)
fmap
```

```
збережено lr5_маршрут.html
по вулицях 50.27 км проти 46.06 км напряму: пряма недооцінює маршрут на 8%
```

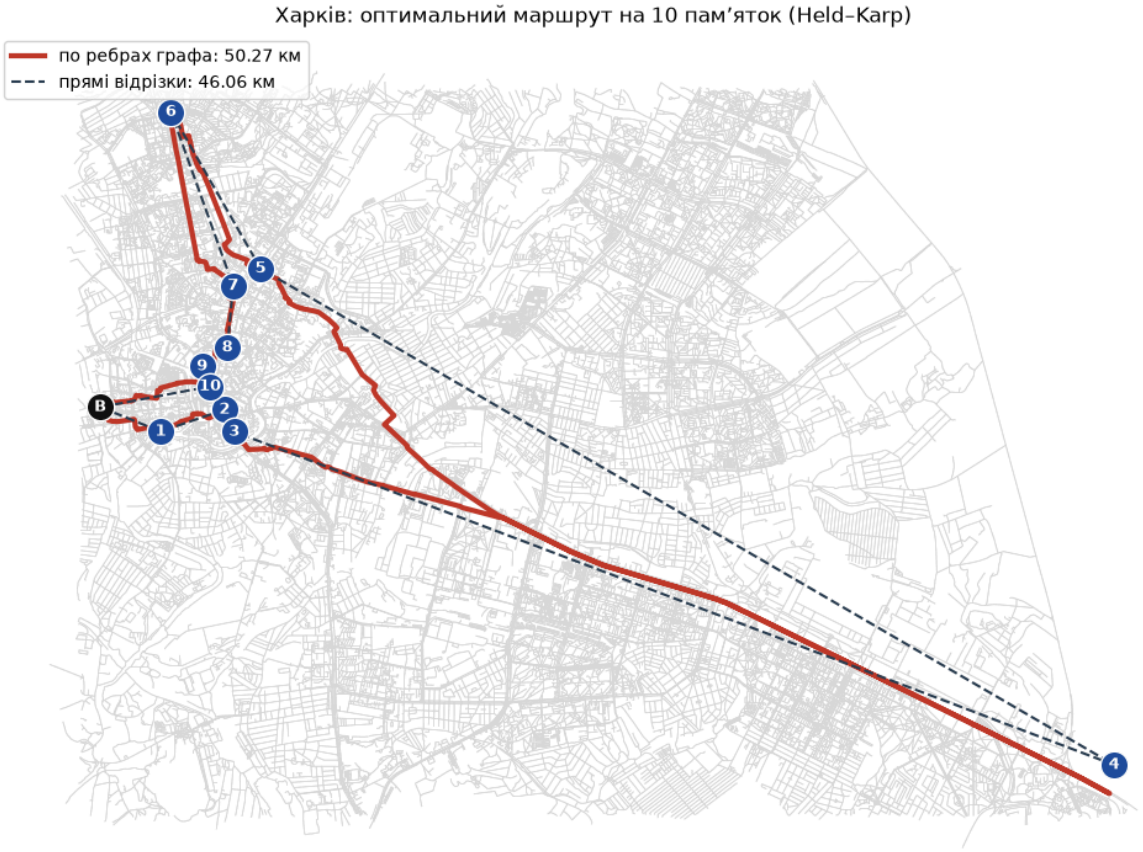
Out[34]:



Хід виконання. Та сама карта будується як звичайний малюнок, щоб різницю між маршрутом вулицями і прямими було видно в PDF. На тлі сірим показано пішохідну мережу.

In [35]:

```
# Те саме статично — щоб різницю було видно і в PDF/збереженому ноутбуці
fig, ax = plt.subplots(figsize=(10, 9))
edges_view = edges.cx[min(c[1] for c in road10) - 0.004: max(c[1] for c in r
                      min(c[0] for c in road10) - 0.003: max(c[0] for c in r
edges_view.plot(ax=ax, color='#d9d9d9', linewidth=0.5)
ax.plot([c[1] for c in road10], [c[0] for c in road10], color='#c0392b', lw=
        label='по ребрах графа: %.2f км' % (L_opt / 1000))
ax.plot([c[1] for c in straight10], [c[0] for c in straight10], color='#2c3e
        label='прямі відрізки: %.2f км' % (L_straight10 / 1000))
for step, i in enumerate(ex10[:-1]):
    x, y = pts10.geometry.x.iloc[i], pts10.geometry.y.iloc[i]
    ax.scatter(x, y, s=260, color='#111' if step == 0 else '#1f4e9c', zorder
    ax.text(x, y, 'B' if step == 0 else str(step), color='white', ha='center
            fontsize=9, fontweight='bold', zorder=6)
ax.set_aspect(1 / np.cos(np.radians(np.mean([c[0] for c in road10]))))
ax.legend(loc='upper left'); ax.set_axis_off()
ax.set_title('%s: оптимальний маршрут на 10 пам'яток (Held–Karp)' % city_wor
plt.tight_layout(); plt.show()
```



---

10. Результати для лабораторної роботи № 8

Збережіть файл, який ЛР8 прочитає і спробує побити.

Хід виконання. Результати зберігаються у файл `lr5_результати.json` для лабораторної роботи № 8: найкращий маршрут на 10 точок, результати наближених методів, координати 30 точок і повна таблиця відстаней між ними.

In [36]:
```
result = {
    'студент': STUDENT_NAME,
    'номер': LIST_NUMBER,
    'місто': CITY,
    'мала_задача': {
        'точок': len(ids10),
        'оптимум_м': round(L_opt, 1),
        'оптимальний_порядок': [str(pts10['name'].iloc[i]) for i in ex10],
        'найближчий_сусід_м': round(tour_length(D10, nn10), 1),
        'двоопт_м': round(tour_length(D10, op10), 1),
    },
    'велика_задача': {
        'точок': len(ids30),
        'найближчий_сусід_м': round(L_nn30, 1),
        'двоопт_м': round(L_op30, 1),
        'двоопт_с': round(t_op30, 4),
```

```
        'точки': [{'name': str(pts30['name'].iloc[i]),
                   'lat': float(pts30.geometry.y.iloc[i]),
                   'lon': float(pts30.geometry.x.iloc[i])} for i in range(le
        'матриця_віддалей_м': [[None if not np.isfinite(v) else round(float(
                                for v in row] for row in D30],
    },
}
with open('lr5_результати.json', 'w', encoding='utf-8') as f:
    json.dump(result, f, ensure_ascii=False, indent=1)
print('збережено lr5_результати.json (%.1f КБ)' % (os.path.getsize('lr5_резу
```

```
збережено lr5_результати.json (17.0 КБ)
```

---

11. Звіт

Відповіді пишіть просто в наступній комірці, числами зі своїх обчислень.

1. **Скільки об'єктів ви відкинули** на кроці 2 і за яким правилом? Чи погодились би ви віддати цей фільтр замовнику без пояснень?
2. **Які сигнали ввійшли у ваш рейтинг і з якими вагами?** Чому саме так?
3. **Скільки об'єктів збіглося** у ваших двох рейтингах із десяти? Що це означає для слова «найкращі»?
4. **Який у вас коефіцієнт обходу** і чи типовий він? Якщо ні — що в місті його пояснює?
5. **На скільки відсотків** найближчий сусід гірший за точний розв'язок? Чи здивувало вас це число?
6. **Чому точний розв'язок на 30 точках недоступний?** Назвіть конкретну величину, яка цьому заважає.
7. **Що у ваших даних виявилось неправильним** — дублікати, пропущені назви, об'єкти не на місці? Як ви з цим вчинили?

Декларація про використання штучного інтелекту

Зазначте, чи користувались ШІ-помічником, у яких саме місцях і як перевіряли отриманий код. Використання дозволене; **неперевірене** використання — ні.

Хід виконання. Звіт містить відповіді на сім питань із числами з обчислень, висновок, якого не було в завданні, і декларацію про використання ШІ.

In [37]:
```
REPORT = '''
Бобринок Ангеліна, ШІ-32, номер у списку 3 -> варіант «Харків, Україна» (міс
Дані © OpenStreetMap contributors, ліцензія ODbL.

0. ДЖЕРЕЛО ДАНИХ (завдання 1.2).
   Межа міста прийшла живим запитом до Nominatim: 0,114 МБ за 1,35 с.
   Живий Overpass 01.11.2026 не працював: overpass-api.de відповідав HTTP 50
   давали таймаут, osm.jp — помилку SSL (і з VPN, і без нього, див. probe_mi
```

```
   private.coffee віддав шар пам'яток (0,52 МБ) приблизно за 29 хвилин і на
   Тому решту шарів узято з резервного знімка — витягу OSM Харківської облас
   download.openstreetmap.fr), з тими самими тегами і тим самим фільтром wal
   Перевірка джерела: пам'яток з pbf 807, з Overpass 796, спільних за (тип,
   даних (частину об'єктів перемальовано з точки на полігон), а не межа: усі
   за 1,4–6 км від межі міста.
   Перший запуск: 126,3 с (завантаження pbf 54,5 МБ за 42,2 с + вирізання 67
   Повторний запуск зі знімка: 0,21 с, тобто приблизно в 600 разів швидше. М
   Окремо для мережі: 14,4 с проти 0,067 с (у 215 разів). Порівняно з Overpa
   29 хв на один шар проти 0,016 с. Контрольна ознака виконана: при повторно
   спосіб «знімок», а передано_МБ = 0.

1. ВІДБІР. У шарі 807 об'єктів, усі мають historic або tourism з переліку (з
   тегами не відкинуто нічого. Відкинуто 95 безіменних (серед них 33 viewpoi
   може бути «якась оглядова точка») і ще 110 меморіальних дошок (historic=m
   дошка на стіні — не місце, куди йдуть пішки. Лишилось 602 кандидати, є і
   Замовнику без пояснень я б цей фільтр НЕ віддала: він пропускає головні г
   собор (Q4369380) і Успенський собор (Q2381859) мають статтю у Вікіпедії,
   amenity=place_of_worship / building=cathedral, без historic і tourism, то
   взагалі. А «Копія Держпрому» і «Копія Успенського собору» з парку мініатю
   Фільтр за тегами вимірює те, як волонтер заповнив теги, а не цінність об'

2. РЕЙТИНГ. score = 0,35·wiki + 0,30·p_lang + 0,20·visit + 0,10·is_hist + 0,
   wiki (є wikidata/wikipedia, 0/1) — найсильніший сигнал: це визнання ззовн
   p_lang (ранг кількості name:* серед кандидатів, 0..1) — перекладають те,
   до задачі туристичного застосунку, але сигнал «шумніший» за wiki, бо пере
   ентузіаст, тому вага менша. visit (музей / галерея / attraction / замок)
   відвідування: для маршруту це важливо, але сам по собі тип нічого не каже
   слабка ознака історичної природи (її має половина кандидатів, тож вона ма
   бо розмір найслабший сигнал: великим буває і склад, а точкові пам'ятники
   Неперервні ознаки переведено в ранги (rank(pct=True)), інакше площа в тис

3. ПОРІВНЯННЯ КРИТЕРІЇВ. Мій топ-10 і топ-10 за площею до чистки дублікатів
   чистки — на 2 з 10 («Будинок зі шпилем» і меморіал «Дробицький яр»). Топ
   поля, городище, музей залізниці, «Американська гірка», автошкола (!) і за
   критерієм (лише wiki + мови) мій збігся на 2 з 10. Стійкість: якщо прибра
   у моїй десятці лишається лише 2 об'єкти з 10. Тобто склад «найкращих» май
   які обрала я, а не дані. «Найкращі» — властивість критерію; відповідальні
   і замовник має бачити формулу разом зі списком.

4. КОЕФІЦІЄНТ ОБХОДУ: 1,15 на 10 точках і 1,19 на 30; по парах від 1,02 до 1
   Історичний музей -> планетарій, 370 м по прямій і 661 м дорогою (через кв
   типових 1,2–1,6. Причина в тому, що точки рознесені по місту (маршрут 50
   пішохідна мережа Харкова (прямі проспекти, діагоналі, дуже щільна сітка д
   повторює пряму. На коротких парах у центрі коефіцієнт доходить до 1,79. А
   двонапрямна, односторонній рух на неї не діє. Недосяжних пар 0, злиплих т
   і 31 на 31). Пряма лінія недооцінює оптимальний маршрут на 8% (46,06 прот

5. МЕТОДИ. На 10 точках: найближчий сусід 52 431 м, тобто на 4,30% (+2,16 км
   (50 271 м). 2-opt — 50 362 м, гірше лише на 0,18% (+91 м). Мене здивувало
   а те, наскільки мало: 4% — це «добре» для методу без гарантій, і 2-opt ма
   На 30 точках: найближчий сусід 75 768 м, 2-opt 67 648 м (коротше на 10,7%
   сусідом і 2-opt зростає з 4% на 10 точках до 11% на 30: наприкінці жадібн
   незручні точки на околицях.
   Це база для ЛР8.
```

```
6. ЧОМУ НЕМАЄ ТОЧНОГО РОЗВ'ЯЗКУ НА 30 ТОЧКАХ: таблиця Held–Karp має 2^30·30
   258 ГБ при 8 байтах на клітину (515 ГБ разом із таблицею предків, у списк
   більше). На цьому ноутбуці 17 ГБ оперативної пам'яті, у 15 разів менше. С
   кожні +5 точок множать потребу приблизно в 30 разів.

7. ЩО В ДАНИХ НЕПРАВИЛЬНО І ЩО Я ЗРОБИЛА.
   а) Експонати як пам'ятки. Танки, гармати і чотири «Половецькі баби» біля
      об'єкти з tourism=attraction і wikidata. Але wikidata посилається на Т
      експонат: у «Танк Т-34-85» стоїть Q2720752 (стаття про модель танка),
      Q12079969. Сигнал «є стаття» обманюється, і до чистки вони займали 8 м
      об'єкти ближче 60 м — одна зупинка, лишаю той, що має вищий рейтинг (с
      а на звільнене місце піднімається наступний кандидат. Видалено 12 об'є
      дублікати: два «Будинки зі шпилем» за 51 м один від одного і два «Дзер
   б) Пропущені пам'ятки: собори без тегів historic/tourism (див. п. 1). Я ц
      розширювала, бо ТЗ задає саме ці теги. У реальному застосунку я б дода
      з wikidata.
   в) «Дробицький яр» — полігон площею 71 тис. м². Його representative_point
      лісу, за 498 м від найближчого вузла графа, тож маршрут «приходить» не
      там теж за 557 м (медіана по всіх точках маршруту 58 м).
   г) Результат залежить від версії даних. «Будинок Шапара» є в моєму топ-10
      Overpass, його немає взагалі (як і ще 15 об'єктів). Той самий код на з
      маршрут, тому дату знімка треба записувати разом із результатом.
   д) До речі, мережа в резервному знімку Львова, що лежить у теці лаби, авт
      на тестовому прогоні точки злипались і коефіцієнт обходу виходив 1,90.

ВИСНОВОК, ЯКОГО НЕ БУЛО В ЗАВДАННІ. «Піший маршрут» за моїм критерієм має 50
найвартісніших об'єктів лежать у 10 різних комірках H3 (половина всіх пам'ят
в 33 комірках з 228, тобто 14,5%). Рейтинг цінності взагалі не враховує прос
обходу (Held–Karp заощадив 4% проти жадібного) не рятує маршрут, який неможл
Більший виграш дає не кращий алгоритм TSP, а зміна постановки: обмеження на
(orienteering problem — набрати максимум цінності за ≤ 8–10 км) або ширший в

Декларація про використання ШІ:
Користувалась ШІ-помічником (Claude Code): він написав код завдань, модуль p
probe_mirrors.py та допоміг сформулювати звіт. Як перевіряла:
 • код спершу прогнано на тестових даних Львова, потім на Харкові; усі вбудо
   (check_tour, 2-opt ≤ NN, Held–Karp ≤ 2-opt) пройдено;
 • метрика: площі та відстані рахуються в EPSG:32635, ребро H3 = 201 м (очік
   у H3 і folium перевірено порядок (lat, lng) — місто на карті там, де має
 • резервний знімок звірено з шаром, що прийшов від Overpass (791 спільний с
   скопійовано з вихідного коду osmnx 2.1.1;
 • close_pairs перевірено на двох точках за 11 м; check_snap показав 11/11 і
 • API osmnx — features_from_* / graph_from_* (не застарілий geometries_from
 • підозрілі результати (танки в топі, 498 м до вузла, коефіцієнт 1,15) пере
   по тегах OSM, а не прийнято як є.
'''
print(REPORT)
```

Бобринок Ангеліна, ШІ-32, номер у списку 3 -> варіант «Харків, Україна» (місто не змінювалось).
Дані © OpenStreetMap contributors, ліцензія ODbL.

0. ДЖЕРЕЛО ДАНИХ (завдання 1.2).
   Межа міста прийшла живим запитом до Nominatim: 0,114 МБ за 1,35 с.
   Живий Overpass 01.11.2026 не працював: overpass-api.de відповідав HTTP 502, lz4/kumi/private.coffee
   давали таймаут, osm.jp — помилку SSL (і з VPN, і без нього, див. probe_mirrors.py). Єдина вдала спроба:
   private.coffee віддав шар пам'яток (0,52 МБ) приблизно за 29 хвилин і на наступному шарі обірвав з'єднання.
   Тому решту шарів узято з резервного знімка — витягу OSM Харківської області (.osm.pbf від 01.11.2026,
   download.openstreetmap.fr), з тими самими тегами і тим самим фільтром walk, що в osmnx (pbf_snapshot.py).
   Перевірка джерела: пам'яток з pbf 807, з Overpass 796, спільних за (тип, id) 791. Розбіжність — версії
   даних (частину об'єктів перемальовано з точки на полігон), а не межа: усі відмінні об'єкти лежать
   за 1,4–6 км від межі міста.
   Перший запуск: 126,3 с (завантаження pbf 54,5 МБ за 42,2 с + вирізання 67,9 с + побудова шарів ~15 с).
   Повторний запуск зі знімка: 0,21 с, тобто приблизно в 600 разів швидше. Мережею не передано 54,6 МБ.
   Окремо для мережі: 14,4 с проти 0,067 с (у 215 разів). Порівняно з Overpass різниця ще більша:
   29 хв на один шар проти 0,016 с. Контрольна ознака виконана: при повторному запуску всі рядки мають
   спосіб «знімок», а передано_МБ = 0.

1. ВІДБІР. У шарі 807 об'єктів, усі мають historic або tourism з переліку (запит уже фільтрував), тож за
   тегами не відкинуто нічого. Відкинуто 95 безіменних (серед них 33 viewpoint і 26 artwork — у маршруті не
   може бути «якась оглядова точка») і ще 110 меморіальних дошок (historic=memorial + memorial=plaque):
   дошка на стіні — не місце, куди йдуть пішки. Лишилось 602 кандидати, є і Point (516), і Polygon (86).
   Замовнику без пояснень я б цей фільтр НЕ віддала: він пропускає головні пам'ятки міста. Покровський
   собор (Q4369380) і Успенський собор (Q2381859) мають статтю у Вікіпедії, але розмічені лише як
   amenity=place_of_worship / building=cathedral, без historic і tourism, тому до вибірки не потрапили
   взагалі. А «Копія Держпрому» і «Копія Успенського собору» з парку мініатюр (tourism=artwork) потрапили.
   Фільтр за тегами вимірює те, як волонтер заповнив теги, а не цінність об'єкта.

2. РЕЙТИНГ. score = 0,35·wiki + 0,30·p_lang + 0,20·visit + 0,10·is_hist + 0,05·p_area.
   wiki (є wikidata/wikipedia, 0/1) — найсильніший сигнал: це визнання ззовні OSM, незалежне джерело.
   p_lang (ранг кількості name:* серед кандидатів, 0..1) — перекладають те, що шукають іноземці; це близько

до задачі туристичного застосунку, але сигнал «шумніший» за wiki, бо перекладі часто додає один
ентузіаст, тому вага менша. visit (музей / галерея / attraction / замок) — об'єкт призначений для
відвідування: для маршруту це важливо, але сам по собі тип нічого не каже про значущість. is_hist —
слабка ознака історичної природи (її має половина кандидатів, тож вона мало розрізняє). p_area — 0,05,
бо розмір найслабший сигнал: великим буває і склад, а точкові пам'ятники мають площу 0.
Неперервні ознаки переведено в ранги (rank(pct=True)), інакше площа в тисячах м² задавила б 0/1-прапорці.

3. ПОРІВНЯННЯ КРИТЕРІЇВ. Мій топ-10 і топ-10 за площею до чистки дублікатів збіглися на 0 з 10, після
чистки — на 2 з 10 («Будинок зі шпилем» і меморіал «Дробицький яр»). Топ за площею — це меморіальні
поля, городище, музей залізниці, «Американська гірка», автошкола (!) і заводські корпуси. З третім
критерієм (лише wiki + мови) мій збігся на 2 з 10. Стійкість: якщо прибрати вагу visit або is_hist,
у моїй десятці лишається лише 2 об'єкти з 10. Тобто склад «найкращих» май же повністю визначають ваги,
які обрала я, а не дані. «Найкращі» — властивість критерію; відповідальність за список несе аналітик,
і замовник має бачити формулу разом зі списком.

4. КОЕФІЦІЄНТ ОБХОДУ: 1,15 на 10 точках і 1,19 на 30; по парах від 1,02 до 1,79. Найбільший обхід:
Історичний музей -> планетарій, 370 м по прямій і 661 м дорогою (через квартал і площу). Це трохи нижче
типових 1,2–1,6. Причина в тому, що точки рознесені по місту (маршрут 50 км), а на довгих відстанях
пішохідна мережа Харкова (прямі проспекти, діагоналі, дуже щільна сітка доріжок: 65 588 вузлів) майже
повторює пряму. На коротких парах у центрі коефіцієнт доходить до 1,79. Асиметрія 0 м: пішохідна мережа
двонапрямна, односторонній рух на неї не діє. Недосяжних пар 0, злиплих точок 0 (11 вузлів на 11 точок
і 31 на 31). Пряма лінія недооцінює оптимальний маршрут на 8% (46,06 проти 50,27 км).

5. МЕТОДИ. На 10 точках: найближчий сусід 52 431 м, тобто на 4,30% (+2,16 км) гірше за точний Held–Karp
(50 271 м). 2-opt — 50 362 м, гірше лише на 0,18% (+91 м). Мене здивувало не те, що жадібний метод гірший,
а те, наскільки мало: 4% — це «добре» для методу без гарантій, і 2-opt майже до нуля прибирає й ці 4%.
На 30 точках: найближчий сусід 75 768 м, 2-opt 67 648 м (коротше на 10,7%) за 0,005 с. Розрив між найближчим
сусідом і 2-opt зростає з 4% на 10 точках до 11% на 30: наприкінці жадібному методу лишаються
незручні точки на околицях.
Це база для ЛР8.

6. ЧОМУ НЕМАЄ ТОЧНОГО РОЗВ'ЯЗКУ НА 30 ТОЧКАХ: таблиця Held–Karp має 2^30·30

= 3,2·10^10 клітин, тобто
   258 ГБ при 8 байтах на клітину (515 ГБ разом із таблицею предків, у списках Python — ще в кілька разів
   більше). На цьому ноутбуці 17 ГБ оперативної пам'яті, у 15 разів менше. Обмеження — пам'ять, а не час:
   кожні +5 точок множать потребу приблизно в 30 разів.

7. ЩО В ДАНИХ НЕПРАВИЛЬНО І ЩО Я ЗРОБИЛА.
   а) Експонати як пам'ятки. Танки, гармати і чотири «Половецькі баби» біля Історичного музею — окремі
      об'єкти з tourism=attraction і wikidata. Але wikidata посилається на ТИП, а не на конкретний
      експонат: у «Танк Т-34-85» стоїть Q2720752 (стаття про модель танка), у всіх чотирьох баб однаковий
      Q12079969. Сигнал «є стаття» обманюється, і до чистки вони займали 8 місць у топ-10. Вирішила так:
      об'єкти ближче 60 м — одна зупинка, лишаю той, що має вищий рейтинг (сам музей), решту видаляю,
      а на звільнене місце піднімається наступний кандидат. Видалено 12 об'єктів, у тому числі справжні
      дублікати: два «Будинки зі шпилем» за 51 м один від одного і два «Дзеркальні струмені» за 19 м.
   б) Пропущені пам'ятки: собори без тегів historic/tourism (див. п. 1). Я це зафіксувала, але фільтр не
      розширювала, бо ТЗ задає саме ці теги. У реальному застосунку я б додала amenity=place_of_worship
      з wikidata.
   в) «Дробицький яр» — полігон площею 71 тис. м². Його representative_point лежить посеред меморіального
      лісу, за 498 м від найближчого вузла графа, тож маршрут «приходить» не до входу. Найближчий заклад
      там теж за 557 м (медіана по всіх точках маршруту 58 м).
   г) Результат залежить від версії даних. «Будинок Шапара» є в моєму топ-10, але в шарі, який віддав
      Overpass, його немає взагалі (як і ще 15 об'єктів). Той самий код на знімку іншої дати дасть інший
      маршрут, тому дату знімка треба записувати разом із результатом.
   д) До речі, мережа в резервному знімку Львова, що лежить у теці лаби, автомобільна, а не пішохідна:
      на тестовому прогоні точки злипались і коефіцієнт обходу виходив 1,90.

ВИСНОВОК, ЯКОГО НЕ БУЛО В ЗАВДАННІ. «Піший маршрут» за моїм критерієм має 50,3 км, бо десять
найвартісніших об'єктів лежать у 10 різних комірках НЗ (половина всіх пам'яток міста зосереджена лише
в 33 комірках з 228, тобто 14,5%). Рейтинг цінності взагалі не враховує простору, тож оптимізація порядку
обходу (Held–Karp заощадив 4% проти жадібного) не рятує маршрут, який неможливо пройти за день.
Більший виграш дає не кращий алгоритм TSP, а зміна постановки: обмеження на довжину маршруту
(orienteering problem — набрати максимум цінності за ≤ 8–10 км) або ширший вибір у щільному центрі.

Декларація про використання ШІ:
Користувалась ШІ-помічником (Claude Code): він написав код завдань, модуль р

bf_snapshot.py і
probe_mirrors.py та допоміг сформулювати звіт. Як перевіряла:
 • код спершу прогнано на тестових даних Львова, потім на Харкові; усі вбудо
вані перевірки
   (check_tour, 2-opt ≤ NN, Held–Karp ≤ 2-opt) пройдено;
 • метрика: площі та відстані рахуються в EPSG:32635, ребро H3 = 201 м (очік
увано ~200 м);
   у H3 і folium перевірено порядок (lat, lng) — місто на карті там, де має
бути;
 • резервний знімок звірено з шаром, що прийшов від Overpass (791 спільний о
б'єкт з ~800), а фільтр walk
   скопійовано з вихідного коду osmnx 2.1.1;
 • close_pairs перевірено на двох точках за 11 м; check_snap показав 11/11 і
31/31 різних вузлів;
 • API osmnx — features_from_* / graph_from_* (не застарілий geometries_from
_place);
 • підозрілі результати (танки в топі, 498 м до вузла, коефіцієнт 1,15) пере
вірено вручну
   по тегах OSM, а не прийнято як є.